

بازیابی اطلاعات

دانشگاه آزاد شیراز - دانشکده مهندسی کامپیوتر

ترم دوم سال تحصیلی ۱۴۰۴-۱۴۰۵

فصل نوزدهم: Web search basics

استاد: دکتر امین اسکندری

تیم طراحی محتوای آموزشی و ویراستاری (علمی و ادبی): حمید نامجو، امیرحسین همتی و علی مجاهد

فهرست مطالب

19.1 Background and history

19.2 Web characteristics

19.2.1 The web graph

19.2.1 Spam

19.3 Advertising as the economic model

19.4 The search user experience

19.4.1 User query needs

19.5 Index size and estimation

19.6 Near-duplicates and shingling

19.7 Chapter 19 Summary

19 و 19.1 مبانی جستجوی وب: پیشینه و تاریخچه

وب جهانی: یک اکوسیستم بی نظیر

وب جهانی (Web) از سه جهت بی سابقه است: (۱) مقیاس عظیم، (۲) عدم هماهنگی در ایجاد محتوا، و (۳) تنوع بی نظیر نویسندگان (از افراد عادی تا سازمان های بزرگ).

برای درک این موضوع، تصور کنید یک کتابخانه‌ی سنتی را با یک بازار بزرگ و شلوغ مقایسه می‌کنید. در کتابخانه، همه چیز سازمان‌یافته، طبقه‌بندی شده و توسط متخصصان مدیریت می‌شود. اما در بازار، هر فروشنده (وب‌سایت) هر چیزی که بخواهد را به هر شکلی که بخواهد ارائه می‌دهد، بدون هماهنگی با دیگران. این دقیقاً همان چیزی است که وب را از مجموعه‌های سنتی اطلاعات متمایز می‌کند و بازیابی اطلاعات در آن را **چالش‌برانگیزتر** می‌سازد.

معماری ساده‌ی وب: چگونه سادگی منجر به پیچیدگی شد؟

طراحی ساده‌ی وب – متشکل از پروتکل **HTTP** و زبان نشانه‌گذاری **HTML** – به صورت عمدی **غیرسخت‌گیرانه** طراحی شد. این سادگی، مانند قوانین ساده‌ی یک زبان جدید بود که به هر کسی اجازه می‌داد بدون نیاز به آموزش فنی، با مشاهده‌ی کد صفحات دیگر، صفحات خود را بسازد.

مثلاً وقتی Tim Berners-Lee اولین مرورگر وب را طراحی کرد، ویژگی "View Source" را اضافه کرد که به کاربران اجازه می‌داد کد HTML صفحات را ببینند. این ویژگی ساده، مانند این بود که یک آشپز حرفه‌ای اجازه دهد هر کسی دستور پخت غذاهایش را ببیند و از آن الگو بگیرد. در نتیجه، **انتشار وب به یک فعالیت جمعی** تبدیل شد – نه انحصار برنامه‌نویسان حرفه‌ای.

ویژگی کلیدی دیگر این بود که مرورگرها هر چیزی را که نمی‌فهمند، نادیده می‌گرفتند. این مانند این بود که یک قاضی به جای رد کردن یک سند به خاطر یک خطای جزئی، صرفاً آن بخش را نادیده بگیرد و به بقیه سند توجه کند. این رویکرد به کاربران اجازه داد بدون ترس از "خراب کردن" سیستم، آزمایش و خطا کنند.

جستجو در وب: دو رویکرد اولیه

با انفجار محتوای وب، یک چالش جدید ظهور کرد: چگونه این اقیانوس اطلاعات را قابل کشف کنیم؟ دو رویکرد اولیه برای این هدف عبارت بودند از:

1. **موتورهای جستجوی مبتنی بر فهرست معکوس** (مثل AltaVista): این رویکرد مانند یک کتابدار بود که تمام کلمات تمام کتاب‌ها را فهرست می‌کند تا بتواند سریعاً کتاب‌های مرتبط با هر کلمه را پیدا کند.
2. **دسته‌بندی‌های دستی** (مثل Yahoo!): این رویکرد مانند یک فروشگاه بزرگ بود که محصولات را در قفسه‌های مختلف طبقه‌بندی می‌کند تا مشتریان بتوانند با گشت و گذار در قفسه‌ها، چیزی که نیاز دارند را پیدا کنند.

چالش‌های دسته‌بندی دستی: داستان Yahoo!

هرچند دسته‌بندی‌های دستی در ظاهر شهودی به نظر می‌رسیدند، اما دو مشکل اساسی داشتند:

- **عدم مقیاس‌پذیری:** تصور کنید بخواهید تمام کتاب‌های یک کتابخانه‌ی ملی را دستی طبقه‌بندی کنید. حالا این کتابخانه هر روز هزاران کتاب جدید دریافت کند! این دقیقاً چالشی بود که Yahoo! با آن روبه‌رو بود. با رشد سریع وب، دسته‌بندی دستی میلیون‌ها صفحه وب، نیازمند نیروی انسانی غیرعملی بود.

• **اختلاف در درک معنایی:** تصور کنید دنبال اطلاعات در مورد "سگ‌های نگهبان" هستید. آیا باید به بخش "حیوانات خانگی" بروید یا "امنیت و حفاظت"؟ این مشکل در Yahoo! وجود داشت که کاربر و ویراستار ممکن است تفسیرهای متفاوتی از جایگاه یک موضوع در درخت دسته‌بندی داشته باشند. در واقع، درخت دسته‌بندی Yahoo! به سرعت به بیش از 1000 گره رسید و پیدا کردن مسیر مناسب برای کاربران دشوار شد.

با وجود این چالش‌ها، برخی پروژه‌ها مانند Open Directory Project تلاش کردند با استفاده از داوطلبان متخصص، این رویکرد را زنده نگه دارند، اما محبوبیت کلی دسته‌بندی‌های دستی کاهش یافت.

🚀 نسل اول موتورهای جستجو: غول‌های فناوری در حال شکل‌گیری

در مقابل، موتورهای جستجوی نسل اول مانند AltaVista، Excite و Infoseek با مقیاس‌های بی‌سابقه (ده‌ها میلیون سند) روبه‌رو شدند. این موتورها مانند کارخانه‌های عظیمی بودند که باید میلیون‌ها محصول را هر روز پردازش می‌کردند. آن‌ها با ترکیب صدها ماشین، سیستم‌هایی با **زمان پاسخ زیر ثانیه** ساختند – یک دستاورد فنی چشمگیر در آن زمان.

با این حال، کیفیت نتایج پایین ماند. دلیل آن: محتوای وب توسط هر کسی و با هر انگیزه‌ای (از جمله اسپم و محتوای بی‌کیفیت) ایجاد می‌شود. این مانند این بود که در یک کتابخانه، هر کسی بتواند هر چیزی را بنویسد و در قفسه‌ها قرار دهد، از مقالات علمی گرفته تا تبلیغات و اطلاعات نادرست.

🎯 فراتر از مرتبط بودن: نیاز به سنجش اعتبار

بنابراین، روش‌های کلاسیک بازیابی (مثل شباهت کسینوس) دیگر کافی نبودند. برای جبران این کاستی، نیاز به معیارهایی فراتر از «مرتبط بودن» احساس شد – معیارهایی که **اعتبار یک سند** را بر اساس منبع آن (مثلاً دامنه میزبان) ارزیابی کنند.

برای مثال، وقتی شما در مورد "سرطان" جستجو می‌کنید، آیا می‌خواهید یک مقاله از سایت Mayo Clinic یا یک وبلاگ شخصی ناشناس را در نتایج اول ببینید؟ این ایده، پایه‌های **الگوریتم‌هایی مانند PageRank** (فصل ۲۱) را فراهم کرد که نه تنها مرتبط بودن، بلکه اعتبار و اقتدار منابع اطلاعاتی را نیز در نظر می‌گیرند.

این تحول، نقطه عطفی در تاریخ جستجوی وب بود که موتورهای جستجو را از صرفاً "پیداکننده‌های کلمات کلیدی" به "ارزیاب‌های کیفیت اطلاعات" تبدیل کرد و راه را برای ظهور گوگل به عنوان غالب‌ترین موتور جستجو هموار کرد.

19.2 ویژگی‌های ذاتی وب: چالش‌های پیش روی موتورهای جستجو 🌐

🌐 جهانی بدون مرز: چالش زبانی و ساختاری

ویژگی‌ای که وب را به شدت گسترش داد – **عدم وجود کنترل مرکزی بر ایجاد محتوا** – همان ویژگی‌ای بود که بزرگ‌ترین چالش را برای موتورهای جستجو ایجاد کرد. نویسندگان وب از ده‌ها زبان طبیعی و هزاران لهجه استفاده می‌کنند، که نیازمند **روش‌های متنوع ریشه‌یابی** و پردازش زبانی است.

برای درک این چالش، تصور کنید موتور جستجوی گوگل باید بتواند همزمان با یک مقاله علمی انگلیسی از سایت Nature، یک وبلاگ فارسی درباره آشپزی، یک صفحه تجاری چینی برای فروش الکترونیک، و یک پست توییتر عربی با املا و قواعد غیراستاندارد کار کند. این فقط چند نمونه از تنوع زبانی است که موتورهای جستجو با آن روبرو هستند.

🎨 **هرج و مرج بصری: از طراحی حرفه‌ای تا وب‌سایت‌های شخصی**

انتشار آزادانه محتوا منجر به **هتروژنی شدید** در جنبه‌های کلیدی شد:

- **سبک و دستور زبان:** بسیاری از صفحات فاقد ساختار زبانی شناخته‌شده هستند.
- **قالب‌بندی بصری:** ترکیب‌های بی‌رویه رنگ، فونت و طرح‌بندی که خوانایی را کاهش می‌دهد.
- **محتوای غیرمتنی:** صفحاتی که کاملاً از تصویر ساخته شده‌اند (بدون متن قابل ایندکس).

برای مثال، صفحه اصلی شرکت اپل در سال ۱۹۹۷ تقریباً کاملاً از تصاویر تشکیل شده بود که برای موتورهای جستجوی آن زمان مانند AltaVista یک چالش بزرگ بود، زیرا هیچ متن قابل ایندکسی برای درک محتوای صفحه وجود نداشت. در مقابل، یک وبلاگ شخصی ممکن است حاوی متن فراوانی باشد اما با ساختار و دستور زبان بسیار ضعیف که درک آن برای الگوریتم‌ها دشوار است.

🔍 **حقیقت در مقابل دروغ: چالش اعتماد در وب**

اما مشکل عمیق‌تر، **کیفیت محتوا** است: وب هم‌زمان حاوی **حقیقت، دروغ، تناقض و حدس** است. این امر سؤال کلیدی را مطرح می‌کند: **به کدام صفحات باید اعتماد کرد؟**

برای مثال، اگر جستجو کنید "آیا واکسن کووید بی‌خطر است؟"، ممکن است با نتایج متناقضی روبرو شوید: از مقالات علمی سایت CDC و WHO گرفته تا پست‌های وبلاگ‌هایی که ادعاهای ضدواکسین را ترویج می‌کنند. موتور جستجو باید بتواند بین این منابع تمایز قائل شود و نتایج معتبرتر را در اولویت قرار دهد.

در نشر سنتی، کاربران منابع مورد اعتماد خود را **انتخاب می‌کنند** (مثل The New York Times یا The Wall Street Journal). اما در وب، کاربر بیشتر اوقات **از طریق موتور جستجو با منبع آشنا می‌شود** – بنابراین، مسئولیت انتخاب منبع معتبر به‌طور غیرمستقیم به موتور جستجو واگذار می‌شود. علاوه بر این، «اعتماد» ممکن است **وابسته به کاربر** باشد: صفحه‌ای برای یک فرد قابل اعتماد است، برای دیگری نه.

📊 **صفحات ایستا در مقابل پویا: چالش مقیاس‌پذیری**

موتورهای جستجو همچنین با چالش **مقیاس وب** روبرو هستند. در سال ۱۹۹۵، AltaVista ادعا کرد که حدود **۳۰ میلیون صفحه ایستا** را ایندکس کرده است. صفحه **ایستا** (Static) به صفحه‌ای گفته می‌شود که محتوای آن بین درخواست‌های متوالی تغییر نمی‌کند – حتی اگر هفته‌ای یک‌بار به‌صورت دستی به‌روزرسانی شود. در مقابل،

صفحات پویا (Dynamic) – مثل وضعیت پرواز فرودگاه – به صورت زنده از پایگاه داده تولید می‌شوند و معمولاً در URL آن‌ها کاراکتر `?` دیده می‌شود (مثل `flight?num=AA129/`).

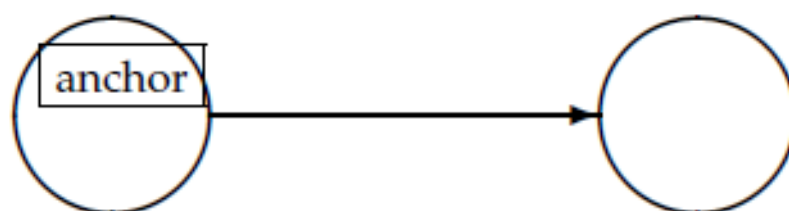
این تقسیم‌بندی برای موتورهای جستجو اهمیت دارد: صفحات پویا دامنه‌ی بی‌پایانی از نتایج ممکن دارند که اندکس کردن آن‌ها نه تنها غیرعملی، بلکه گمراه‌کننده است (چون هر درخواست خروجی متفاوتی تولید می‌کند). بنابراین، موتورهای جستجو معمولاً صفحات ایستا را ترجیح می‌دهند – یا حداقل، مکانیزم‌هایی برای فیلتر کردن صفحات پویای بی‌معنی اعمال می‌کنند.

برای مثال، سایت آمازون میلیون‌ها صفحه پویا دارد که هر کدام ممکن است با تغییر یک پارامتر در URL ایجاد شوند (مثل صفحه محصولات با فیلترهای مختلف قیمت یا رنگ). این در حالی است که یک صفحه شخصی دانشگاهی که هر چند ماه یک‌بار به‌روز می‌شود، یک صفحه ایستا محسوب می‌شود.



► **Figure 19.1** A dynamically generated web page. The browser sends a request for flight information on flight AA129 to the web application, that fetches the information from back-end databases then creates a dynamic web page that it returns to the browser.

شکل 19.1: یک صفحه وب پویا. مرورگر درخواستی برای اطلاعات پرواز AA129 به برنامه وب ارسال می‌کند، برنامه اطلاعات را از پایگاه‌های داده پشتیبان دریافت کرده و یک صفحه وب پویا ایجاد می‌کند که به مرورگر بازمی‌گرداند.



► **Figure 19.2** Two nodes of the web graph joined by a link.

شکل 19.2: دو گره از گراف وب که توسط یک پیوند به هم متصل شده‌اند.

🚀 نتیجه‌گیری: وب به عنوان یک اکوسیستم پیچیده

وب جهانی به دلیل ماهیت غیرمتمرکز و بدون کنترل خود، مجموعه‌ای بی‌نظیر از چالش‌ها را برای موتورهای جستجو ایجاد کرده است. از تنوع زبانی و ساختاری گرفته تا مسائل مربوط به اعتماد و قابلیت اطمینان، همگی نیازمند راه‌حل‌های نوآورانه‌ای هستند که فراتر از تکنیک‌های سنتی بازیابی اطلاعات عمل کنند.

در فصل‌های بعدی، خواهیم دید که موتورهای جستجوی مدرن چگونه با استفاده از الگوریتم‌های پیچیده مانند PageRank و تکنیک‌های یادگیری ماشین، این چالش‌ها را برطرف کرده و توانسته‌اند در این اکوسیستم پیچیده، اطلاعات مرتبط و معتبر را به کاربران ارائه دهند.

19.2.1 گراف وب: شبکه‌ای از ارتباطات

وب به عنوان یک گراف جهت‌دار

وب ایستا را می‌توان به عنوان یک **گراف جهت‌دار** در نظر گرفت:

- **گره‌ها (Nodes):** صفحات HTML.
- **یال‌های جهت‌دار (Directed Edges):** لینک‌های هایپر متنی (hyperlinks).

برای درک بهتر، تصور کنید هر صفحه وب مانند یک ایستگاه در مترو و هر لینک مانند یک خط قطار است که شما را از یک ایستگاه به ایستگاه دیگر منتقل می‌کند. این شبکه‌ی عظیم از ایستگاه‌ها و خطوط، گراف وب را تشکیل می‌دهد.

هر لینک از صفحه A به صفحه B ، یک **out-link** برای A و یک **in-link** برای B محسوب می‌شود. **درجه ورودی** (in-degree) یک صفحه = تعداد لینک‌های ورودی به آن. **درجه خروجی** (out-degree) = تعداد لینک‌های خروجی از آن. مطالعات نشان می‌دهند میانگین درجه ورودی بین ۸ تا ۱۵ است.

برای مثال، صفحه اصلی گوگل میلیون‌ها لینک ورودی از سایت‌های مختلف دارد (in-degree بسیار بالا)، در حالی که وبلاگ شخصی یک دانشجو ممکن است تنها چند لینک ورودی از دوستان و همکلاسی‌هایش داشته باشد (in-degree پایین).

قانون توانی: عدم توازن در محبوبیت

توزیع درجه ورودی از یک **قانون توانی** (Power Law) پیروی می‌کند:

$$i \propto 1 / i^{\alpha} \quad (\alpha \approx 2.1)$$

تعداد صفحات با درجه ورودی

این یعنی تعداد کمی از صفحات (مثل Google یا Wikipedia) **درجه ورودی بسیار بالا** دارند، در حالی که اکثر صفحات تنها چند لینک ورودی دارند.

این پدیده را می‌توان در دنیای واقعی نیز مشاهده کرد: تعداد کمی از سلبریتی‌ها میلیون‌ها دنبال‌کننده دارند، در حالی که اکثر افراد در شبکه‌های اجتماعی تنها چند صد یا چند هزار دنبال‌کننده دارند. این عدم توازن در وب نیز به همین شکل وجود دارد.

ساختار پایبونی وب: چگونه اطلاعات جریان می‌یابد؟

ساختار کلی گراف وب به شکل **Bowtie** (پاپیون) است و سه مؤلفه اصلی دارد:

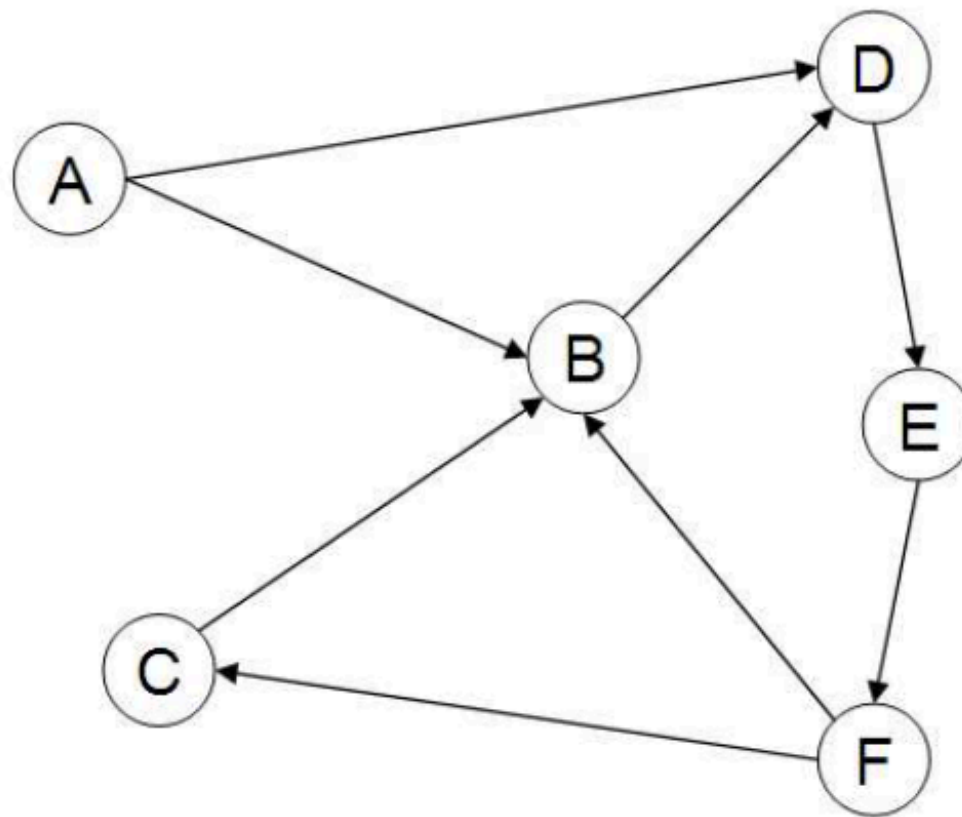
1. **SCC** (Strongly Connected Component): هر صفحه در این بخش به هر صفحه دیگر در همین بخش قابل دسترسی است. این مانند مرکز یک شهر بزرگ است که می‌توانید از هر نقطه‌ای به هر نقطه‌ی دیگر در آن حرکت کنید.

2. **IN**: صفحاتی که فقط به SCC لینک دارند (اما از آن‌ها خروجی به بیرون نیست). این مانند حومه‌های شهری است که فقط مسیرهای ورودی به مرکز شهر دارند.

3. **OUT**: صفحاتی که فقط از SCC لینک دریافت می‌کنند (اما به بیرون لینک نمی‌دهند). این مانند خروجی‌های شهر است که فقط مسیرهای خروجی از مرکز شهر دارند.

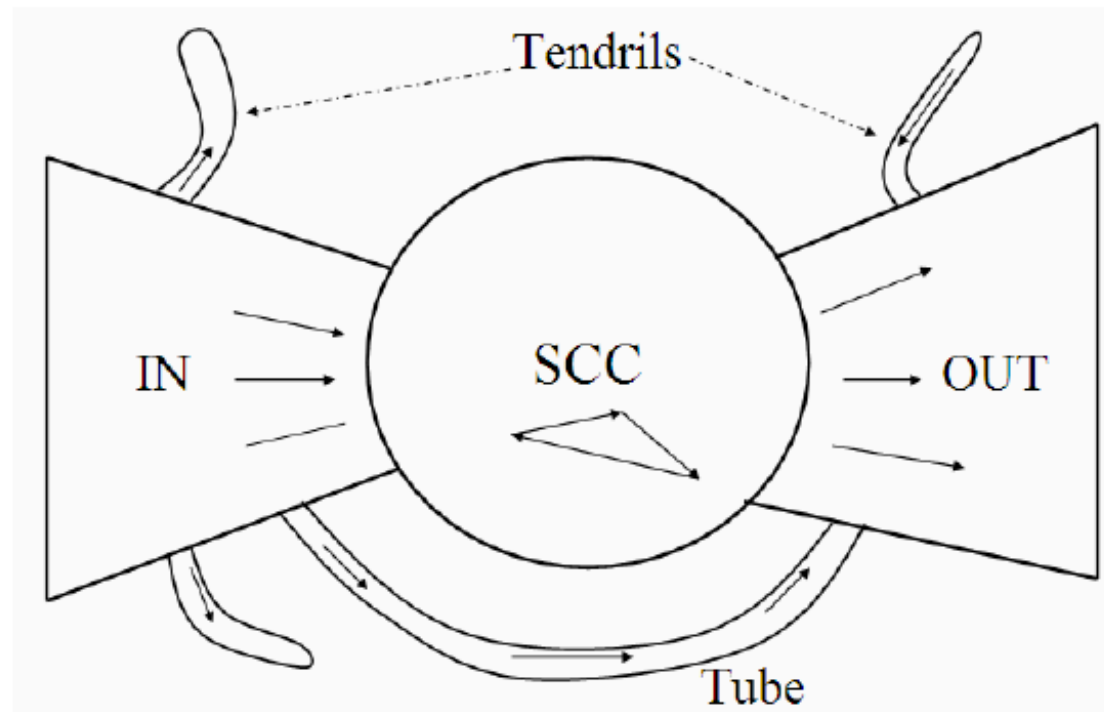
علاوه بر این، بخش‌های کوچکی به نام **tubes** (لوله‌ها) و **tendrils** (رشته‌ها) وجود دارند که به ترتیب مسیرهای مستقیم از IN به OUT و شاخه‌های بن‌بست هستند.

این ساختار نشان می‌دهد که چرا برخی صفحات وب بسیار محبوب‌تر از دیگران هستند: صفحات در SCC مرکز توجه هستند و از هر دو طرف (IN و OUT) ترافیک دریافت می‌کنند، در حالی که صفحات در بخش‌های دیگر ممکن است ترافیک کمتری دریافت کنند.



► **Figure 19.3** A sample small web graph. In this example we have six pages labeled A-F. Page B has in-degree 3 and out-degree 1. This example graph is not strongly connected: there is no path from any of pages B-F to page A.

شکل 19.3: یک نمونه گراف وب کوچک. در این مثال ما شش صفحه با برچسب A-F داریم. صفحه B دارای درجه ورودی 3 و درجه خروجی 1 است. این گراف نمونه قویاً متصل نیست: هیچ مسیری از هیچ یک از صفحات B-F به صفحه A وجود ندارد.



► **Figure 19.4** The bowtie structure of the Web. Here we show one tube and three tendrils.

شکل 19.4: ساختار پاپیونی وب. در اینجا یک لوله و سه رشته را نشان می‌دهیم.

متن لنگر (Anchor Text): پلی بین صفحات

یکی از جنبه‌های مهم گراف وب که در شکل 19.2 نیز نمایش داده شده، **متن لنگر** است. این متنی است که در صفحه مبدأ (A) در اطراف لینک به صفحه مقصد (B) قرار دارد و معمولاً در کد HTML در ویژگی href تگ قرار می‌گیرد.

متن لنگر برای موتورهای جستجو بسیار ارزشمند است، زیرا اغلب توصیفی دقیق از محتوای صفحه مقصد ارائه می‌دهد. برای مثال، اگر بسیاری از سایت‌ها با متن لنگر "بهترین رستوران‌های تهران" به صفحه‌ی شما لینک دهند، موتورهای جستجو احتمالاً صفحه شما را برای این عبارت رتبه‌بندی خواهند کرد.

این ویژگی به شرکت‌هایی مانند گوگل کمک کرده تا الگوریتم‌های بسیار دقیقی برای رتبه‌بندی صفحات توسعه دهندند، زیرا متن لنگر اغلب یک "رأی اعتماد" از طرف صفحه مبدأ به صفحه مقصد محسوب می‌شود.

```
In [1]: import matplotlib.pyplot as plt
import networkx as nx
import numpy as np
import sys

# بازگشتی DFS افزایش محدودیت بازگشت برای پیاده‌سازی
sys.setrecursionlimit(10000)

class WebGraph:
    def __init__(self):
        """
        نمایش گراف وب با دو مدل نگاشت برای دسترسی سریع:
        - out_links: لیست صفحات مقصد (لینک‌های خروجی)
        - in_links: لیست صفحات مبدأ (لینک‌های ورودی)
        این ساختار بهینه برای محاسبات درجه و تحلیل ساختاری است.
        """
        self.out_links = {}
        self.in_links = {}

    def add_page(self, page):
        """افزودن یک صفحه جدید (گره) به گراف بدون لینک اولیه"""
        if page not in self.out_links:
            self.out_links[page] = []
            self.in_links[page] = []

    def add_link(self, from_page, to_page):
        """افزودن یک لینک جهت‌دار از from_page به to_page"""
```

```

است. to_page برای in-link و from_page برای out-link این یک
لینک‌های تکراری اضافه نمی‌شوند.
"""
self.add_page(from_page)
self.add_page(to_page)
if to_page not in self.out_links[from_page]:
    self.out_links[from_page].append(to_page)
    self.in_links[to_page].append(from_page)

def in_degree(self, page):
    """یک صفحه (in-degree) بازگرداندن تعداد لینک‌های ورودی"""
    return len(self.in_links.get(page, []))

def out_degree(self, page):
    """یک صفحه (out-degree) بازگرداندن تعداد لینک‌های خروجی"""
    return len(self.out_links.get(page, []))

def analyze_degrees(self):
    """
    تحلیل توزیع درجه‌های ورودی و خروجی با محاسبه مقادیر آماری
    این توزیع‌ها نشان‌دهنده ساختار قدرت و محبوبیت در وب هستند
    """
    if not self.out_links:
        print("گراف خالی است")
        return

    pages = list(self.out_links.keys())

    in_degrees = [self.in_degree(p) for p in pages]
    out_degrees = [self.out_degree(p) for p in pages]

    print("=== تحلیل توزیع درجه‌ها ===")
    print("میانگین: {:.2f}، انحراف معیار: {:.2f}، حداکثر: {}".format(
        np.mean(in_degrees), np.std(in_degrees), max(in_degrees)))
    print("میانگین: {:.2f}، انحراف معیار: {:.2f}، حداکثر: {}".format(
        np.mean(out_degrees), np.std(out_degrees), max(out_degrees)))
    print("\nجزئیات هر صفحه:")
    for p in sorted(pages):
        print("ورودی: {}, خروجی: {}".format(p, self.in_degree(p), self.out_degree(p)))

def dfs_forward(self, node, visited, order):
    """
    Kosaraju برای مرتب‌سازی توپولوژیکی - پاس اول (DFS) پیمایش عمق‌اول
    """
    visited.add(node)
    for neighbor in self.out_links.get(node, []):
        if neighbor not in visited:
            self.dfs_forward(neighbor, visited, order)
    order.append(node)

def dfs_reverse(self, node, visited, scc):
    """
    Kosaraju روی گراف معکوس - پاس دوم
    """
    visited.add(node)
    scc.add(node)
    for neighbor in self.in_links.get(node, []):
        if neighbor not in visited:
            self.dfs_reverse(neighbor, visited, scc)

def find_all_sccs(self):
    """
    Kosaraju با الگوریتم (SCC) یافتن تمام مؤلفه‌های قویاً همبند
    پیچیدگی زمانی: O(V+E)
    """
    if not self.out_links:
        return []

    # و بر کردن پشته مرتب‌سازی DFS: پاس اول
    visited = set()
    order = []
    for node in self.out_links:
        if node not in visited:
            self.dfs_forward(node, visited, order)

    # order روی گراف معکوس به ترتیب معکوس DFS: پاس دوم
    visited.clear()
    sccs = []
    for node in reversed(order):
        if node not in visited:
            scc = set()
            self.dfs_reverse(node, visited, scc)
            sccs.append(scc)

    return sccs

def analyze_bowtie_structure(self):
    """
    SCC بزرگترین) اصلی SCC بر اساس Bowtie تحلیل ساختار
    این ساختار جریان اطلاعات در وب را به صورت دسته‌بندی شده نشان می‌دهد
    """
    if not self.out_links:
        print("گراف خالی است")
        return

    sccs = self.find_all_sccs()
    if not sccs:

```

```

        print("\n یافت نشد SCC هیچ")
        return

# SCC اصلی: بزرگترین SCC
main_scc = max(sccs, key=len)
all_pages = set(self.out_links.keys())

# قابل دسترسی نیستند SCC برسند اما از SCC می‌توانند به IN بخش
reachable_to_scc = set()
for node in main_scc:
    stack = [node]
    visited = set()
    while stack:
        current = stack.pop()
        if current not in visited:
            visited.add(current)
            for neighbor in self.in_links.get(current, []):
                if neighbor not in visited:
                    stack.append(neighbor)
    reachable_to_scc |= visited
in_set = reachable_to_scc - main_scc

# نمی‌رسند SCC قابل دسترسی‌اند اما به SCC از OUT بخش
reachable_from_scc = set()
for node in main_scc:
    stack = [node]
    visited = set()
    while stack:
        current = stack.pop()
        if current not in visited:
            visited.add(current)
            for neighbor in self.out_links.get(current, []):
                if neighbor not in visited:
                    stack.append(neighbor)
    reachable_from_scc |= visited
out_set = reachable_from_scc - main_scc

# Tubes, Tendrils, Isolated) بقیه صفحات
others = all_pages - (in_set | main_scc | out_set)

# نمایش نتایج
print("\n=== Bowtie تحلیل ساختار ===")
print("SCC (قویاً همبند): {} ({} صفحه)".format(
    sorted(main_scc), len(main_scc)))
print("IN (SCC لینک‌دهنده به): {} ({} صفحه)".format(
    sorted(in_set), len(in_set)))
print("OUT (SCC قابل دسترسی از): {} ({} صفحه)".format(
    sorted(out_set), len(out_set)))
if others:
    print("OTHER (لوله‌ها/ریشه‌های جدا): {} ({} صفحه)".format(
        sorted(others), len(others)))

# توزیع درصدی
total = len(all_pages)
print("\n توزیع درصدی:")
print(" SCC: {:.1%}".format(len(main_scc) / total))
print(" IN: {:.1%}".format(len(in_set) / total))
print(" OUT: {:.1%}".format(len(out_set) / total))
if others:
    print(" OTHER: {:.1%}".format(len(others) / total))

return main_scc, in_set, out_set, others

def demonstrate_power_law_theory(self, max_degree=100, alpha=2.1):
    """
    نمایش تئوری توزیع قانون توان برای درجه ورودی
    دارای لینک‌های بسیار زیادی هستند (Hub) این نشان می‌دهد چرا چند صفحه محبوب
    در حالی که اکثریت صفحات جدا افتاده‌اند.
    """
    degrees = np.arange(1, max_degree + 1)
    probs = 1.0 / (degrees ** alpha)
    probs = probs / probs.sum()

    print("\n=== شبیه‌سازی قانون توان (α={}) ===".format(alpha))
    print("(درجه | احتمال | صفحات مورد انتظار (از ۱۰۰۰))")
    print("-" * 45)
    for i in [1, 2, 3, 5, 10, 20, 50, 100]:
        if i <= max_degree:
            prob = probs[i-1]
            expected = prob * 1000
            print("{:6d} | {:.8f} | {:.2f}".format(i, prob, expected))

# رسم توزیع لگاریتمی برای نمایش
plt.figure(figsize=(10, 6))
plt.loglog(degrees, probs, 'b-', linewidth=2, label='Power Law (α={})'.format(alpha))
plt.xlabel('In-degree (i)', fontsize=12)
plt.ylabel('P(i) (Probability)', fontsize=12)
plt.title('Power Law Distribution of Web Page In-degrees', fontsize=14, fontweight='bold')
plt.grid(True, alpha=0.3, which='both')
plt.legend()
plt.tight_layout()
plt.show()

def plot_actual_degree_distribution(self):
    """
    رسم توزیع درجه‌های واقعی گراف و مقایسه با قانون توان
    """

```

```

if not self.out_links:
    print("گراف خالی است")
    return

pages = list(self.out_links.keys())
in_degrees = [self.in_degree(p) for p in pages]

# محاسبه فراوانی
max_degree = max(in_degrees) if in_degrees else 0
degree_counts = np.zeros(max_degree + 1)
for d in in_degrees:
    degree_counts[d] += 1

# فیلتر کردن درجه‌های صفر برای نمودار لگاریتمی
non_zero = np.arange(1, max_degree + 1)
probs = degree_counts[1:] / len(pages)

plt.figure(figsize=(12, 5))

# نمودار 1: فراوانی
plt.subplot(1, 2, 1)
plt.bar(non_zero, degree_counts[1:], color='skyblue', edgecolor='black')
plt.xlabel('In-degree')
plt.ylabel('Number of Pages')
plt.title('Actual Degree Frequency Distribution')

# نمودار 2: لگاریتمی
plt.subplot(1, 2, 2)
if len(probs) > 0:
    plt.loglog(non_zero, probs, 'ro-', markersize=6, label='Actual Data')
    # خط راهنمای قانون توان
    x = np.linspace(1, max_degree, 100)
    y = 1.0 / (x ** 2.1)
    y = y / sum(y) * sum(probs) # نرمالیزه کردن برای مقایسه
    plt.loglog(x, y, 'b--', label='Power Law ( $\alpha=2.1$ )')

plt.xlabel('In-degree (log scale)')
plt.ylabel('Probability (log scale)')
plt.title('Comparison with Power Law')
plt.legend()
plt.grid(True, alpha=0.3, which='both')

plt.tight_layout()
plt.show()

def visualize(self, layout='spring'):
    """
    Bowtie رسم بصری گراف با رنگ‌بندی خودکار بخش‌های
    """
    if not self.out_links:
        print("گراف خالی است")
        return

    # ایجاد گراف NetworkX
    G = nx.DiGraph()
    G.add_nodes_from(self.out_links.keys())
    for from_page, to_pages in self.out_links.items():
        for to_page in to_pages:
            G.add_edge(from_page, to_page)

    # برای رنگ‌بندی Bowtie تحلیل
    result = self.analyze_bowtie_structure()
    if result is None:
        return

    main_scc, in_set, out_set, others = result

    # رنگ‌بندی خودکار
    color_map = {
        'scc': 'gold',
        'in': 'lightgreen',
        'out': 'lightcoral',
        'other': 'lightgray',
        'default': 'lightblue'
    }

    node_colors = []
    for node in G.nodes():
        if node in main_scc:
            node_colors.append(color_map['scc'])
        elif node in in_set:
            node_colors.append(color_map['in'])
        elif node in out_set:
            node_colors.append(color_map['out'])
        elif node in others:
            node_colors.append(color_map['other'])
        else:
            node_colors.append(color_map['default'])

    # انتخاب الگوریتم چیدمان
    if layout == 'spring':
        pos = nx.spring_layout(G, seed=42, k=2, iterations=50)
    elif layout == 'circular':
        pos = nx.circular_layout(G)
    elif layout == 'kamada':
        pos = nx.kamada_kawai_layout(G)
    else:

```

```

pos = nx.random_layout(G)

fig, ax = plt.subplots(figsize=(12, 8))

# رسم گراف (غیرفعال کردن برجسب‌های پیش‌فرض)
nx.draw(G, pos, with_labels=False, node_color=node_colors,
        node_size=2500, edge_color='gray', linewidths=1.5,
        font_size=11, font_weight='bold', arrowsize=20,
        arrowstyle='->', connectionstyle='arc3,rad=0.1')

# افزودن برجسب‌های سفارشی (فقط یک بار)
labels = {page: f"{page}\n(in={self.in_degree(page)}, out={self.out_degree(page)})"
          for page in G.nodes()}
nx.draw_networkx_labels(G, pos, labels, font_size=9, font_color='black')

# افزودن لیوان رنگ
import matplotlib.patches as mpatches
legend_elements = [
    mpatches.Patch(color=color_map['scc'], label='SCC'),
    mpatches.Patch(color=color_map['in'], label='IN'),
    mpatches.Patch(color=color_map['out'], label='OUT'),
    mpatches.Patch(color=color_map['other'], label='OTHER')
]
ax.legend(handles=legend_elements, loc='upper right')

plt.title("Web Graph Structure with Bowtie Coloring", fontsize=14, fontweight='bold')
plt.axis('off')
plt.tight_layout()
plt.show()

# واقعی SCC نمونه گراف بهبود یافته با
web = WebGraph()

# واقعی SCC ایجاد یک:  $A \leftrightarrow B \leftrightarrow C$ 
web.add_link("A", "B")
web.add_link("B", "C")
web.add_link("C", "A")

# لینک می‌دهند SCC صفحاتی که به IN اضافه کردن
web.add_link("X", "A")
web.add_link("Y", "B")

# قابل دسترسی‌اند SCC صفحاتی که از OUT اضافه کردن
web.add_link("A", "D")
web.add_link("B", "E")
web.add_link("D", "F")

# SCC بدون عبور از OUT به IN مسیر از Tube اضافه کردن یک
web.add_link("Y", "G")
web.add_link("G", "F")

# اضافه کردن یک صفحه منزوی
web.add_page("H")

# تحلیل‌ها
web.analyze_degrees()
web.analyze_bowtie_structure()
web.demonstrate_power_law_theory(max_degree=50, alpha=2.1)

# نمایش داده‌های واقعی و مقایسه با تئوری
web.plot_actual_degree_distribution()

# نمایش بصری
print("\n== Bowtie نمایش بصری با رنگبندی ==")
web.visualize(layout='spring')

```

=== تحلیل توزیع درجه‌ها ===
درجه ورودی: میانگین=1.00، انحراف معیار=0.77، حداکثر=2
درجه خروجی: میانگین=1.00، انحراف معیار=0.77، حداکثر=2

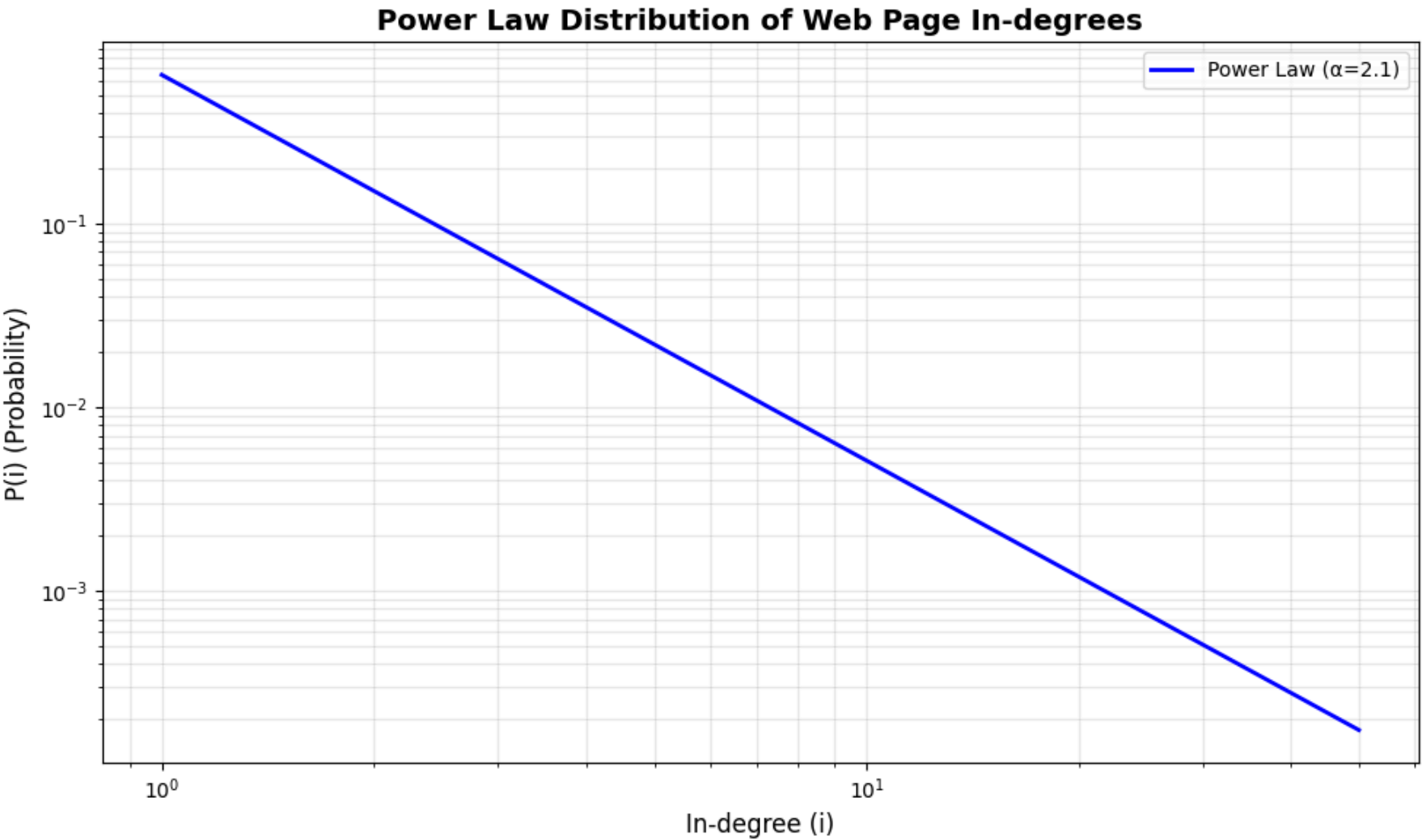
جزئیات هر صفحه:
ورودی=2، خروجی=2 A: صفحه
ورودی=2، خروجی=2 B: صفحه
ورودی=1، خروجی=1 C: صفحه
ورودی=1، خروجی=1 D: صفحه
ورودی=1، خروجی=0 E: صفحه
ورودی=2، خروجی=0 F: صفحه
ورودی=1، خروجی=1 G: صفحه
ورودی=0، خروجی=0 H: صفحه
ورودی=0، خروجی=1 X: صفحه
ورودی=0، خروجی=2 Y: صفحه

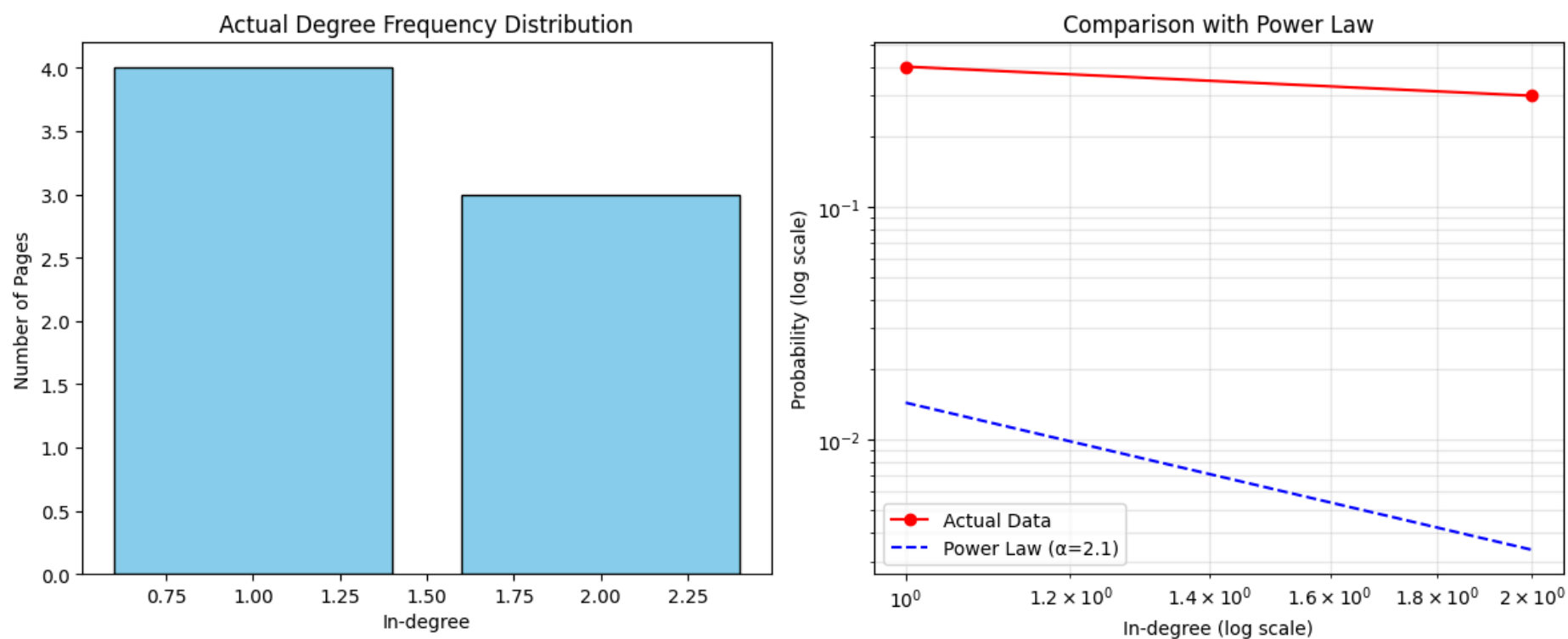
=== Bowtie تحلیل ساختار ===
(صفحه 3) ['A', 'B', 'C']: (قویاً همبند) SCC
(صفحه 2) ['X', 'Y']: (SCC لینک‌دهنده به) IN
(صفحه 3) ['D', 'E', 'F']: (SCC قابل دسترسی از) OUT
(صفحه 2) ['G', 'H']: (لوله‌ها/ریشه‌های جدا) OTHER

توزیع درصدی:
SCC: 30.0%
IN: 20.0%
OUT: 30.0%
OTHER: 20.0%

=== (α=2.1) شبیه‌سازی قانون توان ===
درجه | احتمال | صفحات مورد انتظار (از ۱۰۰۰)

1	0.6460	645.97
2	0.1507	150.68
3	0.0643	64.31
5	0.0220	22.00
10	0.0051	5.13
20	0.0012	1.20
50	0.0002	0.17





=== Bowtie نمایش بصری با رنگبندی ===

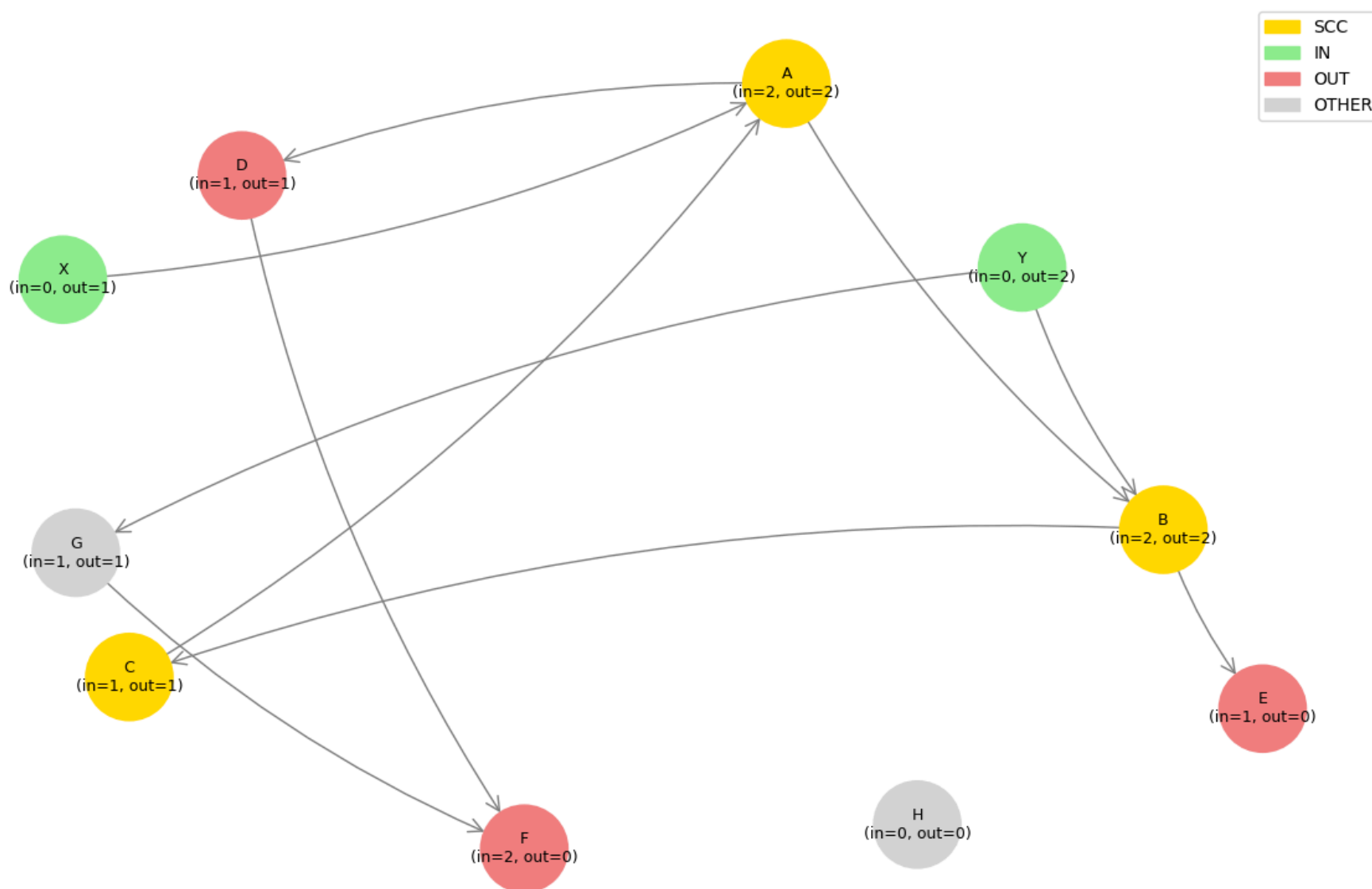
=== تحلیل ساختار Bowtie ===

SCC (صفحه 3): ['A', 'B', 'C'] (قویاً همبند)
 IN (صفحه 2): ['X', 'Y'] (لینکدهنده به)
 OUT (صفحه 3): ['D', 'E', 'F'] (قابل دسترسی از)
 OTHER (صفحه 2): ['G', 'H'] (لوله‌ها/ریشه‌های جدا)

توزیع درصدی:

SCC: 30.0%
 IN: 20.0%
 OUT: 30.0%
 OTHER: 20.0%

Web Graph Structure with Bowtie Coloring



19.2.2 اسپم: جنگ پنهان در جستجوی وب 🕷

💰 **انگیزه اقتصادی: ریشه اصلی اسپم**

از همان ابتدای تاریخ جستجوی وب، مشخص شد که رتبه‌بندی در نتایج جستجو می‌تواند مستقیماً بر فروش تأثیر بگذارد. تصور کنید کاربری عبارت «آپارتمان در سعادت‌آباد» را جستجو می‌کند؛ این کاربر به احتمال زیاد به

دنبال خرید یا اجاره ملک است، نه صرفاً مطالعه درباره این منطقه. همین انگیزه اقتصادی، زمینه را برای بروز اسپم فراهم کرد: دستکاری محتوای صفحات برای رتبه‌بندی بالا در کلمات کلیدی خاص.

این پدیده مشابه شرکت‌هایی است که در دفترچه تلفن‌های قدیمی (Yellow Pages) نام خود را با چند حرف «آ» در ابتدا ثبت می‌کردند تا در صدر فهرست قرار گیرند. اما در دنیای وب، این دستکاری هزینه بسیار کمتری دارد و تأثیر بسیار بیشتری بر نتایج جستجو می‌گذارد.

👤 نسل اول اسپم: تکرار کلمات کلیدی

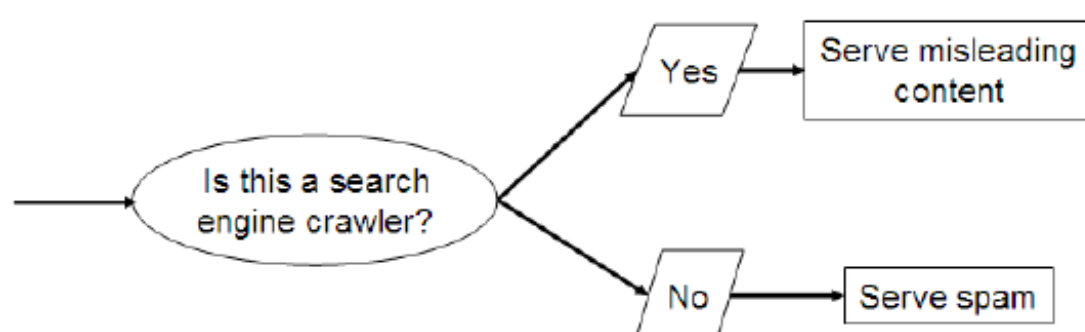
اولین نسل اسپم، تکرار بی‌رویه کلمات کلیدی بود. در موتورهای جستجوی اولیه که بر اساس فراوانی کلمات کار می‌کردند، صفحه‌ای که عبارت «خرید لپ‌تاپ ارزان» را ۱۰۰ بار تکرار می‌کرد، در نتایج جستجو رتبه بالاتری کسب می‌کرد. برای پنهان‌کردن این تکرار از کاربر، اسپمرها از ترفندهایی مانند **رنگ متن مشابه پس‌زمینه** استفاده می‌کردند – که برای انسان نامرئی، اما برای کرالر قابل ایندکس بود. مثلاً در سال‌های اولیه ظهور دیجی‌کالا، رقبا صفحاتی ایجاد می‌کردند که عبارت «خرید موبایل» را با رنگ سفید روی پس‌زمینه سفید تکرار می‌کردند. کاربران این متن را نمی‌دیدند، اما موتورهای جستجو این صفحات را برای این عبارت رتبه‌بندی می‌کردند.

🏠 تکنیک‌های پیشرفته اسپم

با پیشرفت موتورهای جستجو، اسپمرها نیز تکنیک‌های پیچیده‌تری را توسعه دادند:

- **Cloaking (پوشاندن):** سرور، محتوای متفاوتی به کرالر موتور جستجو و کاربر انسانی ارائه می‌دهد. کرالر، صفحه‌ای با کلمات کلیدی فریبنده را ایندکس می‌کند، اما کاربر چیزی کاملاً متفاوت می‌بیند.
- **Doorway Pages (صفحات ورودی):** صفحاتی که فقط برای رتبه‌بندی بالا ساخته شده‌اند و هنگام دسترسی کاربر، وی را به صفحه تجاری واقعی هدایت می‌کنند.
- **Link Spam (اسپم لینک):** ایجاد لینک‌های مصنوعی برای دستکاری معیارهای اعتبارسنجی مبتنی بر گراف وب.

برای مثال، یک شرکت فروش آنلاین ممکن است صفحه‌ای ایجاد کند که برای موتور جستجو پر از کلمات کلیدی «تخفیف ویژه لباس» است، اما وقتی کاربر روی لینک کلیک می‌کند، به صفحه فروشگاه هدایت می‌شود که هیچ ارتباطی با محتوای ایندکس شده ندارد.



► Figure 19.5 Cloaking as used by spammers.

شکل 19.5: تکنیک Cloaking در اسپم. در این روش، سرور محتوای متفاوتی را به کرالر موتور جستجو و کاربر ارائه می‌دهد.

✂ جنگ بین اسپمرها و موتورهای جستجو

این رقابت منجر به شکل‌گیری صنعت بهینه‌سازی موتورهای جستجو (SEO) شد. برخی شرکت‌های SEO به‌صورت قانونی (بهبود محتوا و ساختار سایت) کار می‌کنند، اما برخی دیگر به‌صورت خصمانه تلاش می‌کنند الگوریتم‌های اختصاصی موتورهای جستجو را دور بزنند.

در مقابل، موتورهای جستجو سیاست‌های سخت‌گیرانه‌ای برای مبارزه با اسپم اعمال می‌کنند، از جمله تعلیق کامل وبسایت‌های اسپمر. این جنگ دائمی بین اسپمرها و موتورهای جستجو، زمینه‌ی شکل‌گیری حوزه تحقیقاتی **بازیابی اطلاعات خصمانه (Adversarial IR)** را فراهم کرده است.

برای مثال، گوگل در سال ۲۰۱۲ آپدیت پنگوئن را منتشر کرد که به طور خاص با وبسایت‌هایی که از تکنیک‌های اسپم لینک استفاده می‌کردند مبارزه می‌کرد. این آپدیت باعث شد بسیاری از وبسایت‌هایی که با خرید لینک رتبه خود را artificially بالا برده بودند، افت رتبه شدیدی را تجربه کنند.

🔗 تحلیل لینک: سلاح موتورهای جستجو

یکی از مؤثرترین راه‌های مبارزه با اسپمرها، **تحلیل ساختار لینک‌های وب (Link Analysis)** است. این تکنیک که پایه‌ی الگوریتم‌هایی مانند Google PageRank است، به موتورهای جستجو اجازه می‌دهد تا کیفیت و اعتبار صفحات را بر اساس لینک‌های طبیعی دریافت شده از سایر صفحات ارزیابی کنند.

برای مثال، اگر یک صفحه فروش لپ‌تاپ، لینک‌های طبیعی و باکیفیتی از منابع معتبر مانند زومیت، دیجی‌کالا و وبلاگ‌های تخصصی دریافت کند، احتمالاً در نتایج جستجو رتبه بهتری خواهد داشت، حتی اگر محتوای آن صفحه به اندازه یک صفحه اسپم، کلمات کلیدی را تکرار نکرده باشد.

این رویکرد باعث شده است تا اسپمرها نیز تلاش خود را بر روی دستکاری لینک‌ها متمرکز کنند که به آن **اسپم لینک** می‌گویند. این مبارزه همچنان ادامه دارد و هر دو طرف در حال توسعه تکنیک‌های جدید برای غلبه بر دیگری هستند.

19.3 تبلیغات: مدل اقتصادی که وب را متحول کرد 💰

📊 از بنرهای گرافیکی تا انقلاب تبلیغات کلیکی

در روزهای اولیه وب، شرکت‌ها مانند نایکی و کوکاکولا، بنرهای گرافیکی در سایت‌های محبوبی مانند CNN و Yahoo! قرار می‌دادند. این تبلیغات بر اساس مدل **CPM** (هزینه به ازای هر هزار نمایش) قیمت‌گذاری می‌شدند و هدف اصلی، ایجاد حس مثبت در مخاطب نسبت به برند بود - نه فروش مستقیم.

با افزایش تعاملی بودن وب، مدل **CPC** (هزینه به ازای هر کلیک) متولد شد. در این مدل، مانند آنچه امروز در گوگل ادز می‌بینیم، تبلیغ‌دهنده فقط زمانی هزینه می‌پردازد که کاربر واقعاً روی تبلیغ کلیک کند. این تغییر، تمرکز

را از «تبلیغ برند» به «ایجاد تراکنش» منتقل کرد.

برای درک تفاوت، تصور کنید دیجی‌کالا بتری در سایت ورزش سه نمایش می‌دهد که هر هزار بار نمایش ۱۰۰ هزار تومان هزینه دارد (CPM). در مقابل، وقتی در گوگل «خرید گوشی سامسونگ» را جستجو می‌کنید، تبلیغات نمایش داده‌شده فقط در صورت کلیک هزینه دارند (CPC).

🚀 انقلاب Goto: وقتی جستجو به منبع درآمد تبدیل شد

شرکت **Goto** (که بعداً به Overture تغییر نام داد و در نهایت توسط یاهو خریداری شد)، ایده‌ای انقلابی ارائه داد: برای هر عبارت جستجو، از شرکت‌ها پیشنهاد مالی دریافت می‌کرد و نتایج را بر اساس میزان پیشنهاد رتبه‌بندی می‌کرد.

برای مثال، وقتی کاربری عبارت «هتل مشهد» را جستجو می‌کرد، هتل‌هایی که برای این عبارت بیشتر پیشنهاد داده بودند، در نتایج بالاتر نمایش داده می‌شدند. این مدل به دو دلیل بسیار مؤثر بود:

- کاربران با جستجوی عبارت خاص، **قصد خرید فعال** نشان می‌دادند.
- تبلیغ‌دهندگان فقط برای نتایج واقعی هزینه می‌پرداختند.

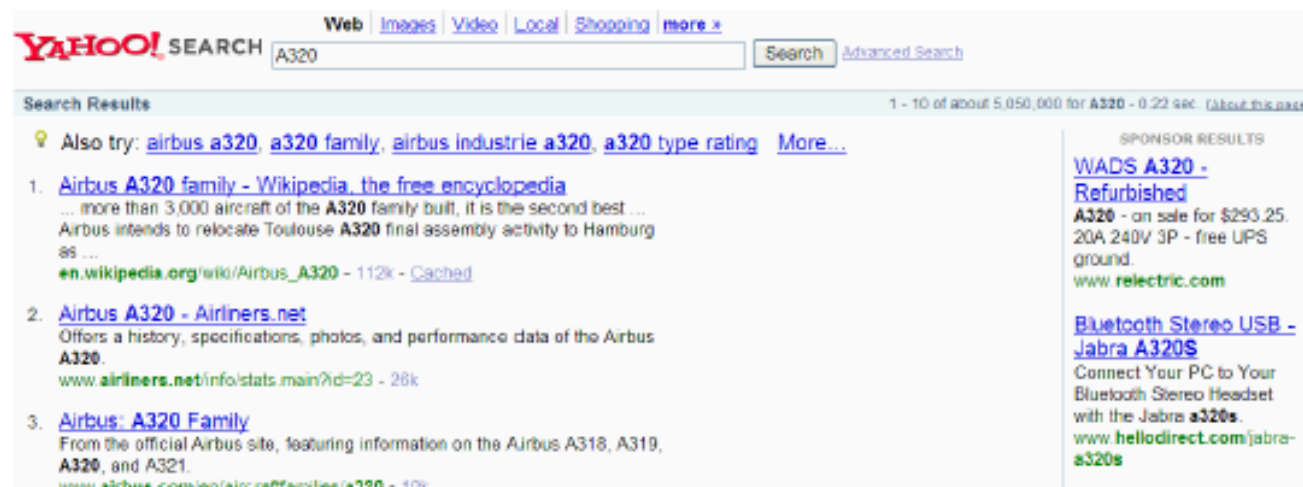
این مدل به سرعت درآمد سالانه Goto/Overture را به صدها میلیون دلار افزایش داد و پایه و اساس مدل تبلیغات مدرن در موتورهای جستجو را شکل داد.

🔍 موتورهای جستجوی مدرن: ترکیبی از نتایج الگوریتمی و تبلیغات

امروزه، موتورهای جستجو مانند گوگل دو نوع نتیجه ارائه می‌دهند:

- **نتایج الگوریتمی:** نتایج اصلی که بر اساس معیارهای فنی مانند PageRank و ارتباط با عبارت جستجو رتبه‌بندی شده‌اند.
- **نتایج اسپانسرشده:** تبلیغات که معمولاً در بالای صفحه یا در کنار نتایج اصلی نمایش داده می‌شوند و با برچسب «تبلیغ» مشخص شده‌اند.

این ترکیب، بهترین هر دو جهان را فراهم می‌کند: کاربران به نتایج مرتبط دسترسی دارند و تبلیغ‌دهندگان می‌توانند به مخاطبان هدف خود دسترسی پیدا کنند.



► **Figure 19.6** Search advertising triggered by query keywords. Here the query A320 returns algorithmic search results about the Airbus aircraft, together with advertisements for various non-aircraft goods numbered A320, that advertisers seek to market to those querying on this query. The lack of advertisements for the aircraft reflects the fact that few marketers attempt to sell A320 aircraft on the web.

شکل 19.6: تبلیغات جستجو triggered by query keywords. در این مثال، جستجوی A320 نتایج الگوریتمی درباره هواپیمای ایرباس A320 را به همراه تبلیغات برای کالاهای غیرمرتبط با هواپیما نمایش می‌دهد که تبلیغ‌کنندگان به دنبال بازاریابی برای کاربران هستند که این عبارت را جستجو می‌کنند.

بازاریابی موتورهای جستجو (SEM): حرفه‌ای نوظهور

با پیچیده‌تر شدن فرآیند رتبه‌بندی تبلیغات در موتورهای جستجو، که ترکیبی از بازاریابی اطلاعات و اقتصاد خرد است، حرفه‌ای جدید به نام **بازاریابی موتورهای جستجو (SEM)** ظهور کرد.

متخصصان SEM به شرکت‌ها کمک می‌کنند تا بودجه بازاریابی خود را بهینه تخصیص دهند، کلمات کلیدی مؤثر را انتخاب کنند و کمپین‌های تبلیغاتی موفق‌تری ایجاد کنند. در ایران نیز شرکت‌هایی مانند وب‌سازان و آژانس‌های دیجیتال مارکتینگ متعددی، خدمات SEM را به کسب‌وکارهای مختلف ارائه می‌دهند.

⚠ چالش کلیک اسپم: وقتی رقابت غیرمنصفانه می‌شود

همان‌طور که هر سیستم اقتصادی با چالش‌هایی روبرو است، تبلیغات در موتورهای جستجو نیز با مشکلی به نام **کلیک اسپم** مواجه است. کلیک اسپم به کلیک‌های غیراصولی روی تبلیغات گفته می‌شود که از طرف کاربران واقعی نیستند.

برای مثال، یک رقیب ممکن است با استفاده از ربات‌ها، به طور مکرر روی تبلیغات شما کلیک کند تا بودجه تبلیغاتی شما سریع‌تر تمام شود. این چالش، موتورهای جستجو را مجبور کرده است الگوریتم‌های پیچیده‌ای برای شناسایی و نادیده گرفتن کلیک‌های اسپم توسعه دهند.

این جنگ میان اسپمرها و موتورهای جستجو، نمونه‌ای دیگر از رقابت خصمانه در اکوسیستم وب است که به «بازاریابی اطلاعات خصمانه» معروف است.

19.4 تجربه کاربری در جستجوی وب: انقلابی در تعامل با اطلاعات

از متخصصان به عموم: تحول کاربران جستجو

در سیستم‌های بازیابی اطلاعات سنتی، کاربران معمولاً متخصصانی بودند که با ساختار مجموعه‌های اطلاعاتی آشنا بودند و در نگارش پرس‌وجوها آموزش دیده بودند. اما در جستجوی وب، با میلیاردها کاربر عادی روبرو هستیم که:

- به‌طور میانگین فقط ۲ تا ۳ کلمه کلیدی در جستجوی خود استفاده می‌کنند
- به ندرت از عملگرهای پیچیده (مانند AND, OR, *) استفاده می‌کنند
- درباره ساختار و ناهمگونی محتوای وب اطلاعی ندارند و اهمیتی هم نمی‌دهند

برای درک این تفاوت، تصور کنید یک کتابدار متخصص که با استفاده از دستورات پیچیده در کاتالوگ کتابخانه جستجو می‌کند در مقابل یک دانشجو که در گوگل "نظریه نسبیت اینشتین" را تایپ می‌کند. این تغییر بنیادی، نیازمند طراحی موتورهای جستجویی بود که برای افراد عادی، نه متخصصان، مناسب باشند.

اصول موفقیت گوگل: دقت و سادگی

در رقابت شدید بین موتورهای جستجو، گوگل با دو اصل کلیدی توانست بر رقبای خود پیشی بگیرد:

1. **تمرکز بر دقت (Precision) به جای جامعیت (Recall):** گوگل متوجه شد کاربران عادی به نتایج دقیق در ۳-۵ جایگاه اول اهمیت می‌دهند، نه لیست‌های طولانی از نتایج مرتبط. این اصل باعث صرفه‌جویی در زمان کاربر و افزایش رضایت او می‌شود.

2. **تجربه کاربری سبک و سریع:** در حالی که رقبای گوگل مانند یاهو و AltaVista صفحه‌های شلوغی با محتوای خبری، ایمیل و بنرهای تبلیغاتی داشتند، گوگل صفحه‌ای ساده، متنی و با بارگذاری سریع ارائه داد. این طراحی مینیمالیستی باعث شد کاربران به سرعت به نتیجه برسند.

این دو اصل در کنار هم، ترکیبی قدرتمند ایجاد کردند: کاربران سریع‌تر به اطلاعات دقیق‌تر دسترسی پیدا کردند و این باعث افزایش ترافیک و در نتیجه درآمد گوگل از تبلیغات شد.

سه دسته اصلی پرس‌وجوهای کاربران

تحقیقات نشان داده است که پرس‌وجوهای جستجوی وب را می‌توان در سه دسته اصلی طبقه‌بندی کرد:

پرس‌وجوهای ناوبری (Navigational)

کاربر به دنبال وب‌سایت یا صفحه خاصی است که از قبل نام آن را می‌داند و انتظار دارد نتیجه اول همان سایت باشد.

پرس‌وجوهای اطلاعاتی (Informational)

کاربر به دنبال اطلاعات عمومی درباره یک موضوع است و معمولاً از چندین منبع استفاده می‌کند.

مثال‌های واقعی:

- "علائم بیماری کووید-۱۹ چیست؟"
- "تاریخچه جنگ جهانی دوم"
- "آموزش پخت کیک شکلاتی"

مثال‌های واقعی:

- "فیس‌بوک"
- "دیجی‌کالا"
- "بانک ملی"

پرس‌وجوهای تراکنشی (Transactional)

کاربر قصد انجام یک عمل آنلاین مانند خرید، دانلود یا رزرو را دارد و به دنبال سایتی برای انجام این تراکنش است.

مثال‌های واقعی:

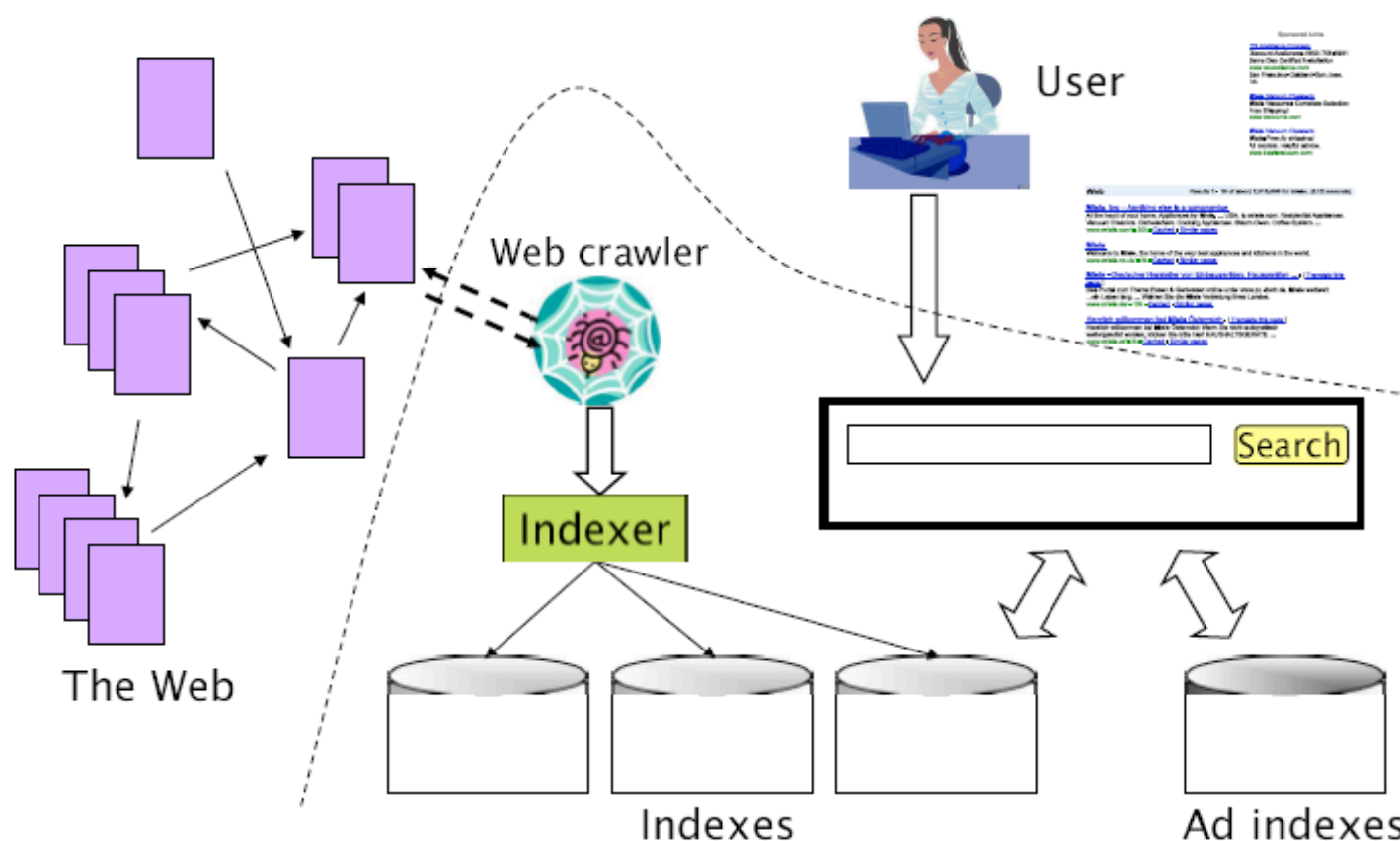
- "خرید گوشی آیفون ۱۴"
- "دانلود نرم‌افزار فتوشاپ"
- "رزرو هتل در مشهد"

اهمیت تشخیص نوع پرس‌وجو

درک نوع پرس‌وجو برای موتورهای جستجو از دو جهت حیاتی است:

1. **بهبود نتایج الگوریتمی:** برای پرس‌وجوهای ناوبری، دقت در رتبه اول ($Precision@1$) مهم‌ترین شاخص موفقیت است. وقتی کاربر "یوتوب" را جستجو می‌کند، انتظار دارد اولین نتیجه، سایت اصلی یوتیوب باشد.
2. **بهینه‌سازی تبلیغات:** پرس‌وجوهای تراکنشی اغلب نشان‌دهنده نیت خرید هستند و بنابراین برای نمایش تبلیغات اسپانسرشده ایده‌آل هستند. برای مثال، وقتی کاربر "خرید لپ‌تاپ ایسوس" را جستجو می‌کند، نمایش تبلیغات فروشگاه‌های مختلف لپ‌تاپ بسیار مؤثرتر از نمایش تبلیغات برای مقالات علمی درباره لپ‌تاپ است.

این درک عمیق از نیازهای کاربران، موتورهای جستجو را قادر می‌سازد تا تجربه‌ای شخصی‌سازی‌شده و مفید ارائه دهند که کاربران را به استفاده مداوم از سرویس خود تشویق کند.



► **Figure 19.7** The various components of a web search engine.

شکل 19.7: تصویر ترکیبی از یک موتور جستجوی وب شامل کرالر، ایندکس صفحات وب و ایندکس تبلیغات. بخش زیر خط منحنی ممتد داخلی موتور جستجو است.

19.5 اندازه ایندکس: چالش‌های اندازه‌گیری وب نامحدود

چرا اندازه‌گیری ایندکس پیچیده است؟ 🌐

در نگاه اول، ممکن است فکر کنید اندازه ایندکس یک موتور جستجو معیار ساده‌ای برای ارزیابی جامعیت آن است. اما واقعیت بسیار پیچیده‌تر است. وب شامل تعداد نامحدودی صفحات پویا است که هر بار با بارگذاری مجدد، محتوای جدیدی تولید می‌کنند.

برای درک این چالش، به این مثال توجه کنید: آدرس `http://www.amazon.com/search?keyword=any_string` برای هر عبارت دلخواهی که جایگزین `any_string` شود، یک صفحه معتبر برمی‌گرداند - حتی اگر محصولی با آن نام وجود نداشته باشد. این صفحات "Soft 404" که به جای خطای 404، صفحه‌ای معتبر نمایش می‌دهند، فضای جستجو را به صورت تئوری نامحدود می‌کنند.

علاوه بر این، برخی سایت‌ها عمداً "دام‌های کرالر" (Spider Traps) ایجاد می‌کنند - صفحاتی که کرالرهای موتورهای جستجو را در حلقه‌ای از لینک‌های بی‌پایان گرفتار می‌کنند تا ایندکس آن‌ها را با صفحات بی‌ارزش پر کنند.

روش Capture-Recapture: از زیست‌شناسی تا موتورهای جستجو 🦋

به جای اندازه مطلق، معمولاً نسبت اندازه ایندکس دو موتور جستجو (مثلاً گوگل در مقابل بینگ) مورد علاقه است. برای تخمین این نسبت، محققان از روشی به نام **Capture-Recapture** استفاده کرده‌اند که اصل آن از

زیست‌شناسی گرفته شده است.

در زیست‌شناسی، برای تخمین جمعیت ماهیان در یک دریاچه، زیست‌شناسان ابتدا تعدادی ماهی صید و علامت‌گذاری می‌کنند (Capture). سپس ماهیان را آزاد کرده و در صید دوم، تعداد ماهیان علامت‌دار در نمونه جدید را می‌شمارند (Recapture). با این اطلاعات، می‌توانند تخمین بزنند که چند درصد از کل جمعیت در نمونه اول حضور داشته‌اند.

در دنیای موتورهای جستجو، این روش به صورت زیر عمل می‌کند:

- نمونه‌ای از ایندکس موتور E_1 انتخاب می‌شود و بررسی می‌شود چه کسری از آن در E_2 وجود دارد $x \rightarrow$
- نمونه‌ای از ایندکس E_2 انتخاب می‌شود و بررسی می‌شود چه کسری در E_1 وجود دارد $y \rightarrow$

با این فرضیات، نسبت اندازه ایندکس‌ها به صورت زیر تخمین زده می‌شود:

$$|E_1| / |E_2| \approx y / x$$

البته این روش با فرض‌های ساده‌کننده‌ای کار می‌کند که در عمل کاملاً دقیق نیستند: اینکه هر موتور جستجو یک زیرمجموعه تصادفی و مستقل از وب را ایندکس می‌کند.

روش‌های نمونه‌برداری و چالش‌های آن‌ها

در عمل، نمونه‌برداری واقعی از ایندکس موتورهای جستجو چالش‌برانگیز است. چندین روش پیشنهاد شده است، اما هر کدام با سوگیری‌های خاص خود همراه است:

آدرس‌های IP تصادفی

در این روش، آدرس‌های IP تصادفی تولید شده و به سرورهای وب درخواست ارسال می‌شود. این روش با مشکلاتی مانند میزبانی مجازی (چند سایت در یک IP) و سوگیری به سمت سایت‌های کوچک مواجه است.

جستجوهای تصادفی

در این روش، از لاگ جستجوی کاربران واقعی استفاده می‌شود و یک جستجوی تصادفی انتخاب می‌شود. مشکل این روش این است که نمونه‌برداری به سمت علایق گروه خاصی از کاربران (مثلاً محققان یک دانشگاه) سوگیری دارد.

قدم‌زدن تصادفی (Random Walk)

این روش بر پایه حرکت تصادفی در گراف وب عمل می‌کند، اما با این چالش که وب یک گراف قویاً همبند نیست و همگرایی به توزیع پایا بسیار کند است.

? پرس‌وجوهای تصادفی

رایج‌ترین روش، استفاده از پرس‌وجوهای تصادفی با کلمات هم‌ایستا است. این روش نیز با سوگیری به سمت اسناد بلند و تأثیر الگوریتم رتبه‌بندی موتور جستجو مواجه است.

برای مثال، اگر برای نمونه‌برداری از گوگل از پرس‌وجوی تصادفی استفاده کنیم، ممکن است به طور ناخواسته به سمت صفحاتی سوگیری داشته باشیم که گوگل آن‌ها را رتبه بالاتری داده است، نه نمونه‌ای واقعاً تصادفی از کل ایندکس.

🇮🇷 پیشرفت‌های اخیر در تخمین اندازه ایندکس

پژوهش‌های اخیر سعی کرده‌اند این سوگیری‌ها را با روش‌های آماری پیشرفته‌تر جبران کنند. یکی از رویکردهای نوین، **نمونه‌برداری با قدم‌زدن تصادفی در گراف مجازی اسناد** است.

در این روش، یک گراف مجازی ایجاد می‌شود که در آن گره‌ها اسناد هستند و دو سند با یک یال به هم متصل می‌شوند اگر حداقل دو کلمه مشترک داشته باشند. سپس یک قدم‌زدن تصادفی در این گراف انجام می‌شود بدون اینکه گراف به طور کامل ساخته شود.

این رویکردهای جدید، اگرچه کامل نیستند، اما درک دقیق‌تری از اندازه نسبی ایندکس موتورهای جستجو ارائه می‌دهند. این موضوع نشان می‌دهد که حتی ساده‌ترین سؤالات درباره وب، پاسخ‌های پیچیده‌ای دارند و نیازمند ترکیبی از دانش از حوزه‌های مختلف مانند آمار، علوم کامپیوتر و نظریه گراف هستیم.

```
In [15]: import matplotlib.pyplot as plt
import numpy as np

# -----
# Capture-Recapture شبیه‌سازی روش
# برای تخمین نسبت اندازه ایندکس موتورهای جستجو
# -----

class SearchEngine:
    """
    نمایش ساده یک موتور جستجو
    pages: مجموعه صفحاتی که ایندکس شده‌اند
    """
    def __init__(self, name, pages):
        self.name = name
        self.pages = set(pages)

    def sample(self, n):
        """نمونه‌برداری یکنواخت (فرض ایده‌آل)"""
        return np.random.choice(list(self.pages), min(n, len(self.pages)), replace=False)

    def contains(self, page):
        """بررسی وجود صفحه در ایندکس"""
        return page in self.pages

def capture_recapture(engine1, engine2, sample_size):
    """
    تخمین نسبت اندازه ایندکس:  $|E1| / |E2|$ 

    فرمول:
     $ratio = (E2 \text{ در نمونه } E1 \text{ درصد صفحات}) / (E1 \text{ در نمونه } E2 \text{ درصد صفحات})$ 
    """
    # E2 و بررسی وجود در E1 نمونه‌گیری از
    sample_e1 = engine1.sample(sample_size)
    overlap_in_e2 = sum(p in engine2.pages for p in sample_e1)
    pct_e1_in_e2 = overlap_in_e2 / len(sample_e1)

    # E1 و بررسی وجود در E2 نمونه‌گیری از
    sample_e2 = engine2.sample(sample_size)
    overlap_in_e1 = sum(p in engine1.pages for p in sample_e2)
    pct_e2_in_e1 = overlap_in_e1 / len(sample_e2)

    # محاسبه نسبت
    if pct_e1_in_e2 == 0:
        return float('inf'), pct_e1_in_e2, pct_e2_in_e1

    ratio = pct_e2_in_e1 / pct_e1_in_e2
    return ratio, pct_e1_in_e2, pct_e2_in_e1

# -----
# سناریوی شبیه‌سازی
```

```
# -----

# برای نتایج قابل تکرار seed تنظیم
np.random.seed(42)

# ایجاد یک وب سنتتیک: 100,000 صفحه
all_pages = [f"page_{i}" for i in range(100000)]

# ایندکس 60% وب - Google: موتور جستجو 1
google = SearchEngine("Google", all_pages[:60000])

# Google ایندکس 40% وب با 30% همپوشانی با - Bing: موتور جستجو 2
bing = SearchEngine("Bing", all_pages[30000:70000])

# نسبت واقعی (برای مقایسه)
true_ratio = len(google.pages) / len(bing.pages)

print("شبیهسازی روش Capture-Recapture")
print("=" * 45)
print(f"|Google|/|Bing|: {true_ratio:.3f}")
print(f"تعداد صفحات Google: {len(google.pages):,}")
print(f"تعداد صفحات Bing: {len(bing.pages):,}")
print(f"تعداد صفحات مشترک: {len(google.pages & bing.pages):,}")
print()

# -----
# بررسی با اندازه‌های نمونه مختلف
# -----

sample_sizes = [100, 500, 1000, 5000, 10000]
estimates = []

for n in sample_sizes:
    ratio, x, y = capture_recapture(google, bing, n)
    error = abs(ratio - true_ratio) / true_ratio * 100
    estimates.append((ratio, error))
    print(f"نمونه: {n:6d} | تخمین: {ratio:6.3f} | خطا: {error:5.2f}% | x={x:.3f} y={y:.3f}")

# -----
# نمایش نتایج به صورت بصری
# -----

ratios = [e[0] for e in estimates]
errors = [e[1] for e in estimates]

plt.figure(figsize=(12, 5))

# نمودار 1: نسبت تخمین‌شده
plt.subplot(1, 2, 1)
plt.plot(sample_sizes, ratios, 'bo-', linewidth=2, markersize=6, label='Capture-Recapture Estimation')
plt.axhline(y=true_ratio, color='r', linestyle='--', linewidth=2, label=f'Real Ratio: {true_ratio:.3f}')
plt.xscale('log')
plt.xlabel('Sample Size (log scale)', fontsize=11)
plt.ylabel('Estimated Ratio |E1| / |E2|', fontsize=11)
plt.title('Capture-Recapture Estimation', fontsize=12, fontweight='bold')
plt.legend(fontsize=10)
plt.grid(True, which='both', ls='--', alpha=0.5)

# نمودار 2: خطای درصدی
plt.subplot(1, 2, 2)
plt.plot(sample_sizes, errors, 'ro-', linewidth=2, markersize=6, label='Estimation Error')
plt.xscale('log')
plt.xlabel('Sample Size (log scale)', fontsize=11)
plt.ylabel('Error (%)', fontsize=11)
plt.title('Estimation Error vs Sample Size', fontsize=12, fontweight='bold')
plt.legend(fontsize=10)
plt.grid(True, which='both', ls='--', alpha=0.5)

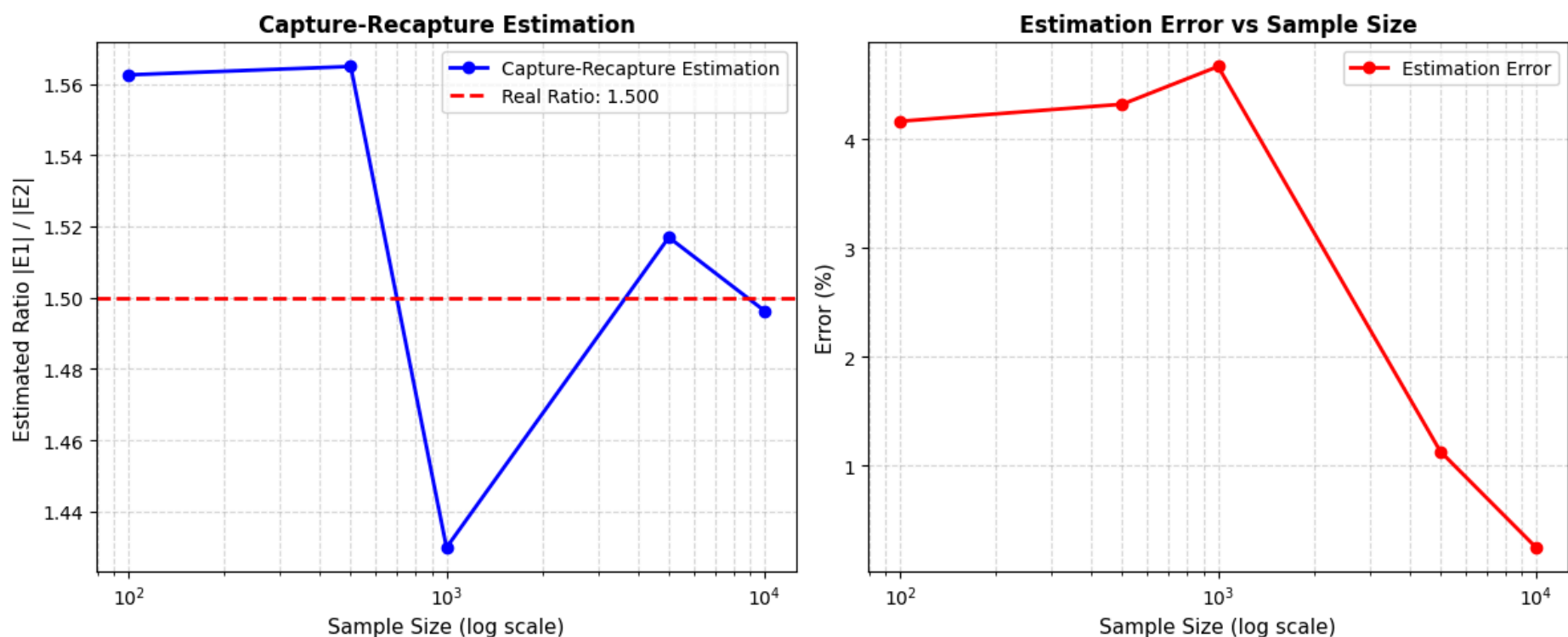
plt.tight_layout()
plt.show()

print("\n" + "=" * 60)
print("نکات کلیدی:")
print("1. افزایش اندازه نمونه → خطای تخمین کمتر.")
print("2. این روش فرض می‌کند نمونه‌برداری یکنواخت از ایندکس ممکن است.")
print("3. در واقعیت، نمونه‌برداری از طریق پرس‌وجوهای تصادفی انجام می‌شود.")
print("4. پرس‌وجوها سوگیری دارند (مثلاً به سمت صفحات بلند یا محبوب).")
print("5. نیاز است (Horvitz-Thompson مثل) برای اصلاح سوگیری به روش‌های آماری پیشرفته.")
print("نکته: نتایج برای نمونه‌های کوچک دارای واریانس بالایی هستند #")
```

Capture-Recapture شبیهسازی روش
=====

|Google|/|Bing|: 1.500
تعداد صفحات Google: 60,000
تعداد صفحات Bing: 40,000
تعداد صفحات مشترک: 30,000

4.17	خطا:		1.562	تخمین:		100	%نمونه:		x=0.480	y=0.750
4.32	خطا:		1.565	تخمین:		500	%نمونه:		x=0.478	y=0.748
4.67	خطا:		1.430	تخمین:		1000	%نمونه:		x=0.521	y=0.745
1.13	خطا:		1.517	تخمین:		5000	%نمونه:		x=0.490	y=0.744
0.25	خطا:		1.496	تخمین:		10000	%نمونه:		x=0.503	y=0.752



نکات کلیدی:

- افزایش اندازه نمونه → خطای تخمین کمتر
- این روش فرض می‌کند نمونه‌برداری یکنواخت از ایندکس ممکن است
- در واقعیت، نمونه‌برداری از طریق پرس‌وجوهای تصادفی انجام می‌شود
- پرس‌وجوها سوگیری دارند (مثلاً به سمت صفحات بلند یا محبوب)
- نیاز است (مثلاً Horvitz-Thompson) برای اصلاح سوگیری به روش‌های آماری پیشرفته

نکته: نتایج برای نمونه‌های کوچک دارای واریانس بالایی هستند #

🔄 19.6 صفحات تقریباً تکراری و روش Shingling

🌐 چالش تکرار در وب

تخمین اندازه ایندکس بدون در نظر گرفتن **تکرارها** گمراه‌کننده است: تا **۴۰٪ صفحات وب تکراری یا تقریباً تکراری (Near-Duplicate)** هستند. این پدیده دلایل مختلفی دارد:

- **میرورهای قانونی:** سایت‌های خبری مانند CNN یا BBC محتوای خود را در چند سرور مختلف برای افزایش دسترسی و قابلیت اطمینان کپی می‌کنند.
- **محتوای پویا:** صفحاتی که فقط در بخش‌های کوچکی مانند زمان آخرین به‌روزرسانی یا شمارنده بازدیدکنندگان تفاوت دارند.
- **سامانه‌های مدیریت محتوا:** قالب‌های استاندارد که باعث ایجاد صفحات با ساختار بسیار مشابه اما محتوای کمی متفاوت می‌شوند.

برای مثال، خبر یک رویداد ورزشی در سایت‌های مختلف اغلب با متن اصلی یکسان است، اما با عناوین متفاوت یا با افزودن تاریخ و زمان انتشار. موتورهای جستجو باید بتوانند این صفحات را به‌عنوان نسخه‌های مختلف یک محتوای واحد شناسایی کنند تا از ایندکس کردن چندباره آن جلوگیری شود.

🔍 از اثر انگشت تا Shingling

برای تشخیص تکرارهای دقیق، می‌توان از **اثر انگشت (Fingerprint)** استفاده کرد - یک خلاصه ۶۴-بیتی از کل محتوای صفحه. اما این روش برای صفحاتی که فقط در چند کاراکتر تفاوت دارند، شکست می‌خورد.

برای حل این مشکل، از تکنیک **Shingling** استفاده می‌شود. در این روش، هر سند به‌عنوان مجموعه‌ای از **k-shingle** در نظر گرفته می‌شود که دنباله‌های پیوسته از k واژه در سند هستند.

برای مثال، متن خبری زیر را در نظر بگیرید:

"تیم ملی فوتبال ایران امروز در دیداری حساس به مصاف ژاپن رفت و با نتیجه ۲ بر یک به پیروزی رسید."

اگر $k=4$ باشد، 4-shingleهای این متن عبارتند از:

"تیم ملی فوتبال"، "ملی فوتبال ایران"، "فوتبال ایران امروز"، "ایران امروز در"، "امروز در دیداری"، "در دیداری حساس"، "دیداری حساس به"، "حساس به مصاف"، "به مصاف ژاپن"، "مصاف ژاپن رفت"، "ژاپن رفت و"، "رفت و با نتیجه"، "و با نتیجه ۲"، "با نتیجه ۲ بر"، "نتیجه ۲ بر یک"، "۲ بر یک به"، "بر یک به پیروزی"، "یک به پیروزی رسید"

حالا اگر خبر دیگری با متن زیر وجود داشته باشد:

"تیم ملی فوتبال ایران امروز در دیداری حساس به مصاف ژاپن رفت و با نتیجه ۲ بر یک به پیروزی رسید. این پیروزی در تاریخ ۱۴۰۲/۰۲/۱۰ ثبت شد."

این دو متن در اکثر shingleها مشترک هستند و بنابراین به عنوان صفحات تقریباً تکراری شناسایی می شوند.

ضرب جاکارد و الگوریتم کارآمد

برای اندازه گیری شباهت بین مجموعه shingleهای دو سند، از **ضرب جاکارد** استفاده می شود:

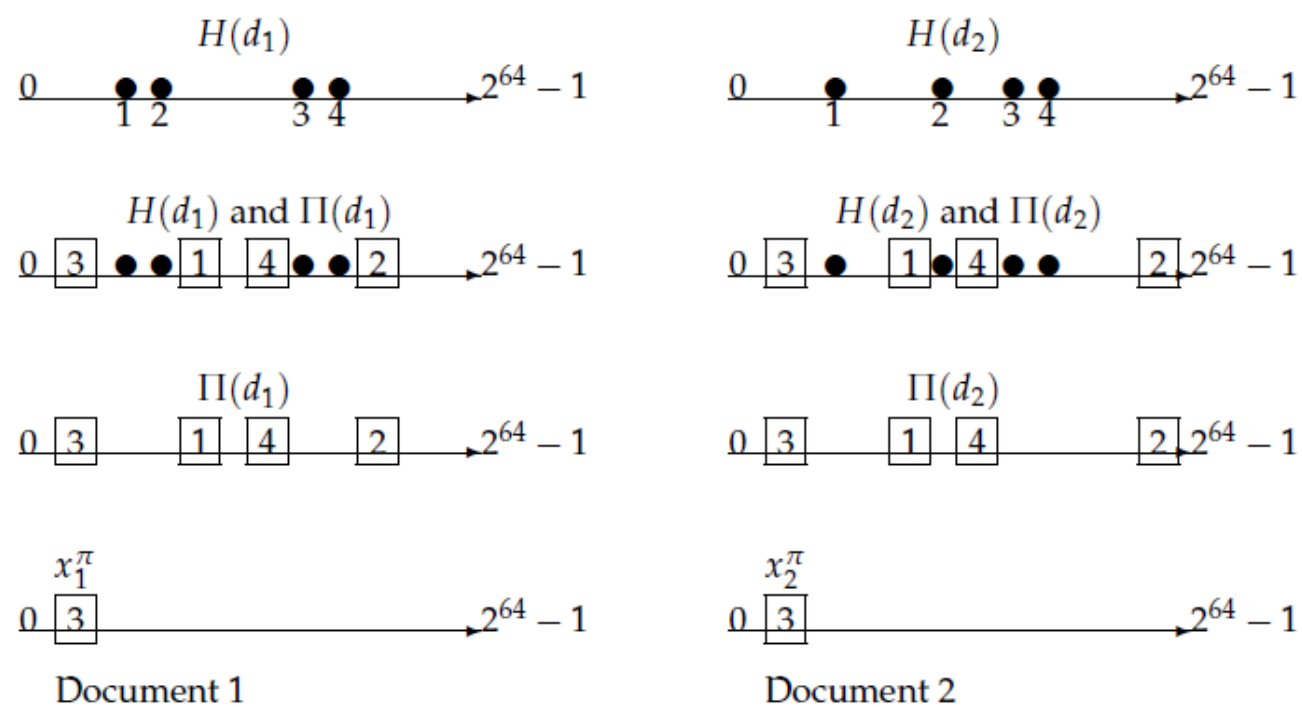
$$J(A, B) = |A \cap B| / |A \cup B|$$

اما محاسبه این ضرب برای میلیاردها سند غیرعملی است. راه حل، استفاده از تکنیک های مبتنی بر **درهم سازی (Hashing)** و **جایگشت تصادفی (Random Permutation)** است:

- هر shingle را به یک عدد ۶۴-بیتی درهم سازی می کنیم.
- یک جایگشت تصادفی روی فضای درهم سازی اعمال می کنیم.
- کوچک ترین مقدار درهم سازی شده پس از جایگشت را به عنوان **امضا (Signature)** سند ذخیره می کنیم.

یک نتیجه شگفت انگیز (قضیه ۱۹.۱) نشان می دهد که احتمال برابری امضاهای دو سند، دقیقاً برابر با ضرب جاکارد آنهاست!

در عمل، این کار را برای **۲۰۰ جایگشت مستقل** تکرار می کنیم تا یک **اسکچ (Sketch)** از ۲۰۰ عدد برای هر سند بسازیم. سپس تعداد اشتراک اسکچ های دو سند را می شماریم و اگر این مقدار از آستانه ای (مثلاً ۱۶۰ از ۲۰۰) بیشتر بود، صفحات را تقریباً تکراری می دانیم.



► **Figure 19.8** Illustration of shingle sketches. We see two documents going through four stages of shingle sketch computation. In the first step (top row), we apply a 64-bit hash to each shingle from each document to obtain $H(d_1)$ and $H(d_2)$ (circles). Next, we apply a random permutation Π to permute $H(d_1)$ and $H(d_2)$, obtaining $\Pi(d_1)$ and $\Pi(d_2)$ (squares). The third row shows only $\Pi(d_1)$ and $\Pi(d_2)$, while the bottom row shows the minimum values x_1^π and x_2^π for each document.

شکل 19.8: تصویرسازی از محاسبه اسکچ shingle. دو سند در چهار مرحله پردازش می‌شوند: ابتدا هر shingle به یک مقدار ۶۴-بیتی درهم‌سازی می‌شود (دایره‌ها)، سپس یک جایگشت تصادفی اعمال می‌شود (مربع‌ها)، در مرحله سوم فقط مقادیر جایگشت‌یافته نمایش داده می‌شود، و در نهایت کوچک‌ترین مقادیر برای هر سند مشخص می‌شود.

⚡ بهینه‌سازی با Super-Shingles

حتی با این روش، مقایسه تمام جفت‌های ممکن از اسناد هنوز بسیار پرهزینه است. برای کاهش هزینه محاسباتی، از تکنیک **Super-Shingles** استفاده می‌شود:

1. اسکچ ۲۰۰ تایی هر سند را مرتب می‌کنیم.
 2. از این اسکچ مرتب‌شده، shingle‌های جدیدی (Super-Shingles) می‌سازیم.
 3. فقط جفت‌های اسنادی که حداقل یک Super-Shingle مشترک دارند را با دقت کامل بررسی می‌کنیم.
- این روش مانند یک فیلتر اولیه عمل می‌کند که اکثر جفت‌های غیرمشابه را حذف می‌کند و فقط به بررسی جفت‌های امیدبخش می‌پردازد. با این بهینه‌سازی، موتورهای جستجو می‌توانند با کارایی بالا صفحات تقریباً تکراری را شناسایی کرده و از ایندکس کردن چندباره آن‌ها جلوگیری کنند.

```
In [19]: import random
import hashlib
import numpy as np
import matplotlib.pyplot as plt

# -----
# برای تشخیص صفحات تقریباً تکراری MinHash و Shingling پیاده‌سازی
# -----

class Shingling:
    """از متن k-shingles استخراج"""
    def __init__(self, k=4):
        self.k = k

    def get_shingles(self, text):
        """k-shingles متن به مجموعه"""
        words = text.split()
        if len(words) < self.k:
            return set()
```

```
        return {tuple(words[i:i+self.k]) for i in range(len(words) - self.k + 1)}

class MinHash:
    """های ثابت permutation با MinHash پیاده‌سازی"""
    def __init__(self, num_permutations=200, seed=42):
        self.num_permutations = num_permutations
        # ها یکبار تولید می‌شوند و ثابت می‌مانند permutation
        random.seed(seed)
        self.permutations = [(random.randint(1, 2**64 - 1), random.randint(0, 2**64 - 1))
                              for _ in range(num_permutations)]

    def _hash_shingle(self, shingle):
        """به عدد 64 بیتی shingle درهم‌سازی یک"""
        shingle_str = " ".join(shingle)
        hash_obj = hashlib.md5(shingle_str.encode())
        return int.from_bytes(hash_obj.digest()[:8], byteorder='little', signed=False)

    def create_signature(self, shingles):
        """shingle برای یک مجموعه MinHash ساخت امضای"""
        if not shingles:
            return [0] * self.num_permutations

        hashed = [self._hash_shingle(s) for s in shingles]
        signature = []

        # های ثابت برای همه اسناد استفاده می‌شود permutation از همان
        for a, b in self.permutations:
            min_hash = min((a * h + b) % (2**64) for h in hashed)
            signature.append(min_hash)

        return signature

    def estimate_similarity(self, sig1, sig2):
        """تخمین ضریب جاکارد از روی دو امضا"""
        matches = sum(1 for a, b in zip(sig1, sig2) if a == b)
        return matches / len(sig1)

class NearDuplicateDetector:
    """Shingling و MinHash تشخیص صفحات تقریباً تکراری با استفاده از"""
    def __init__(self, k=4, threshold=0.7, num_perm=200, seed=42):
        self.k = k
        self.threshold = threshold
        self.shingler = Shingling(k)
        self.minhash = MinHash(num_perm, seed)

    def is_near_duplicate(self, text1, text2):
        """
        تشخیص اینکه آیا دو متن تقریباً تکراری هستند
        تخمینی_jaccard, دقیق_jaccard (boolean, خروجی):
        """

        # محاسبه shingles
        shingles1 = self.shingler.get_shingles(text1)
        shingles2 = self.shingler.get_shingles(text2)

        # محاسبه جاکارد دقیق برای مقایسه
        union = shingles1 | shingles2
        intersection = shingles1 & shingles2
        exact_jaccard = len(intersection) / len(union) if union else 0

        # محاسبه تخمین MinHash
        sig1 = self.minhash.create_signature(shingles1)
        sig2 = self.minhash.create_signature(shingles2)
        est_jaccard = self.minhash.estimate_similarity(sig1, sig2)

        is_duplicate = est_jaccard >= self.threshold
        return is_duplicate, exact_jaccard, est_jaccard

# -----
# سناریوی آموزشی
# -----

# خبر اصلی
news1 = "تیم ملی فوتبال ایران امروز در دیدی حساس به مصاف ژاپن رفت و با نتیجه ۲ بر یک به پیروزی رسید"

# خبر تقریباً تکراری: افزودن تاریخ و زمان
news2 = "تیم ملی فوتبال ایران امروز در دیدی حساس به مصاف ژاپن رفت و با نتیجه ۲ بر یک به پیروزی رسید این پیروزی در تاریخ ۱۴۰۲ ثبت شد"

# خبر کاملاً متفاوت
news3 = "قیمت طلا در بازار جهانی با افزایش ۵ درصدی روبرو شد و به ۲۰۰۰ دلار در هر اونس رسید"

print("MinHash و Shingling تشخیص صفحات تقریباً تکراری با")
print("=" * 60)

# detector تنظیم (k=4, threshold=0.7, 200 permutation)
detector = NearDuplicateDetector(k=4, threshold=0.7, num_perm=200, seed=42)

# مقایسه 1 و 2
is_dup, exact, est = detector.is_near_duplicate(news1, news2)
print(f"مقایسه خبر اصلی و خبر با تاریخ اضافی:")
print(f"تعداد کلمات: {len(news1.split())} vs {len(news2.split())}")
print(f"جاکارد دقیق: {exact:.3f} (|A∩B|/|A∪B|)")
print(f"MinHash تخمین: {est:.3f} ({200} permutation)")
print(f"مقایسه: {'تفاوت' if is_dup else 'تقریباً تکراری'}: نتیجه")
```

```
# مقایسه 1 و 3
is_dup, exact, est = detector.is_near_duplicate(news1, news3)
print(f"\nمقایسه خبر اصلی و خبر متفاوت:")
print(f"جاکارد دقیق: {exact:.3f}")
print(f"MinHash تخمین: {est:.3f}")
print(f"{'تفاوت X' if is_dup else 'تقریباً تکراری ✓'}: نتیجه")

# -----
# ها permutation بررسی تجربی: تاثیر تعداد
# -----

def experiment_permutations():
    """ها، دقت افزایش می‌یابد permutation تایید قضیه ۱۹.۱: با افزایش"""
    perm_counts = [10, 20, 50, 100, 200, 500]
    results = []

    for count in perm_counts:
        detector = NearDuplicateDetector(k=4, threshold=0.7, num_perm=count, seed=42)
        _, exact, est = detector.is_near_duplicate(news1, news2)
        error = abs(est - exact) / exact * 100 if exact > 0 else 0
        results.append((est, error))

    # رسم نمودار
    plt.figure(figsize=(10, 6))
    estimates = [r[0] for r in results]
    errors = [r[1] for r in results]

    plt.plot(perm_counts, estimates, 'bo-', linewidth=2, markersize=6, label='MinHash Estimation')
    plt.axhline(y=exact, color='r', linestyle='--', linewidth=2, label=f'Exact Jaccard: {exact:.3f}')
    plt.xlabel('Number of Permutations', fontsize=12)
    plt.ylabel('Estimated Similarity', fontsize=12)
    plt.title('Theorem 19.1: Permutations vs Estimation Accuracy', fontsize=14, fontweight='bold')
    plt.legend(fontsize=10)
    plt.grid(True, which='both', ls='--', alpha=0.5)
    plt.xscale('log')
    plt.tight_layout()
    plt.show()

    # چاپ نتایج
    print("\nها بر دقت (قضیه ۱۹.۱) permutation تاثیر تعداد:")
    print("=" * 50)
    for count, (est, error) in zip(perm_counts, results):
        print(f"{count:4d} permutations: estimate={est:.3f}, error={error:.2f}%")

# اجرا
experiment_permutations()

# -----
# Super-Shingles تحلیل
# -----

def analyze_super_shingles(band_size=10):
    """
    فیلتر اولیه برای کاهش ۹۹٪ مقایسه‌ها Super-Shingles نمایش کار
    مشترک دارند بررسی می‌شوند Super-Shingle فقط اسنادی که حداقل یک
    """
    detector = NearDuplicateDetector(k=4, threshold=0.7, num_perm=200, seed=42)

    # محاسبه signatures
    sig1 = detector.minhash.create_signature(detector.shingler.get_shingles(news1))
    sig2 = detector.minhash.create_signature(detector.shingler.get_shingles(news2))
    sig3 = detector.minhash.create_signature(detector.shingler.get_shingles(news3))

    # های متوالی band به signature تقسیم Super-Shingles ساخت
    num_bands = len(sig1) // band_size
    super1 = {tuple(sig1[i*band_size:(i+1)*band_size]) for i in range(num_bands)}
    super2 = {tuple(sig2[i*band_size:(i+1)*band_size]) for i in range(num_bands)}
    super3 = {tuple(sig3[i*band_size:(i+1)*band_size]) for i in range(num_bands)}

    # محاسبه اشتراک
    common_12 = len(super1 & super2)
    common_13 = len(super1 & super3)

    print("\n" + "=" * 50)
    print("Super-Shingles بهینه‌سازی با")
    print(f"{num_bands} در هر متن Super-Shingles تعداد:")
    print(f"مشترک {common_12} (خبر ۱ و ۲) Super-Shingles")
    print(f"مشترک {common_13} (خبر ۱ و ۳) Super-Shingles")

    # تصمیم‌گیری بر اساس فیلتر
    print(f"Super-Shingles نتیجه فیلتر:")
    if common_12 > 0:
        print("MinHash مشترک → بررسی دقیق با Super-Shingle خبر ۱ و ۲: دارای ✓")
    else:
        print("مشترک → بدون بررسی دقیق رد می‌شوند Super-Shingle خبر ۱ و ۲: بدون X")

    if common_13 > 0:
        print("مشترک → بررسی دقیق می‌شود Super-Shingle خبر ۱ و ۳: دارای ✓")
    else:
        print("مشترک → بدون بررسی دقیق رد می‌شود Super-Shingle خبر ۱ و ۳: بدون X")

    print("\n💡 نکته: Super-Shingles ۹۹٪ را بدون محاسبه دقیق رد می‌کند")

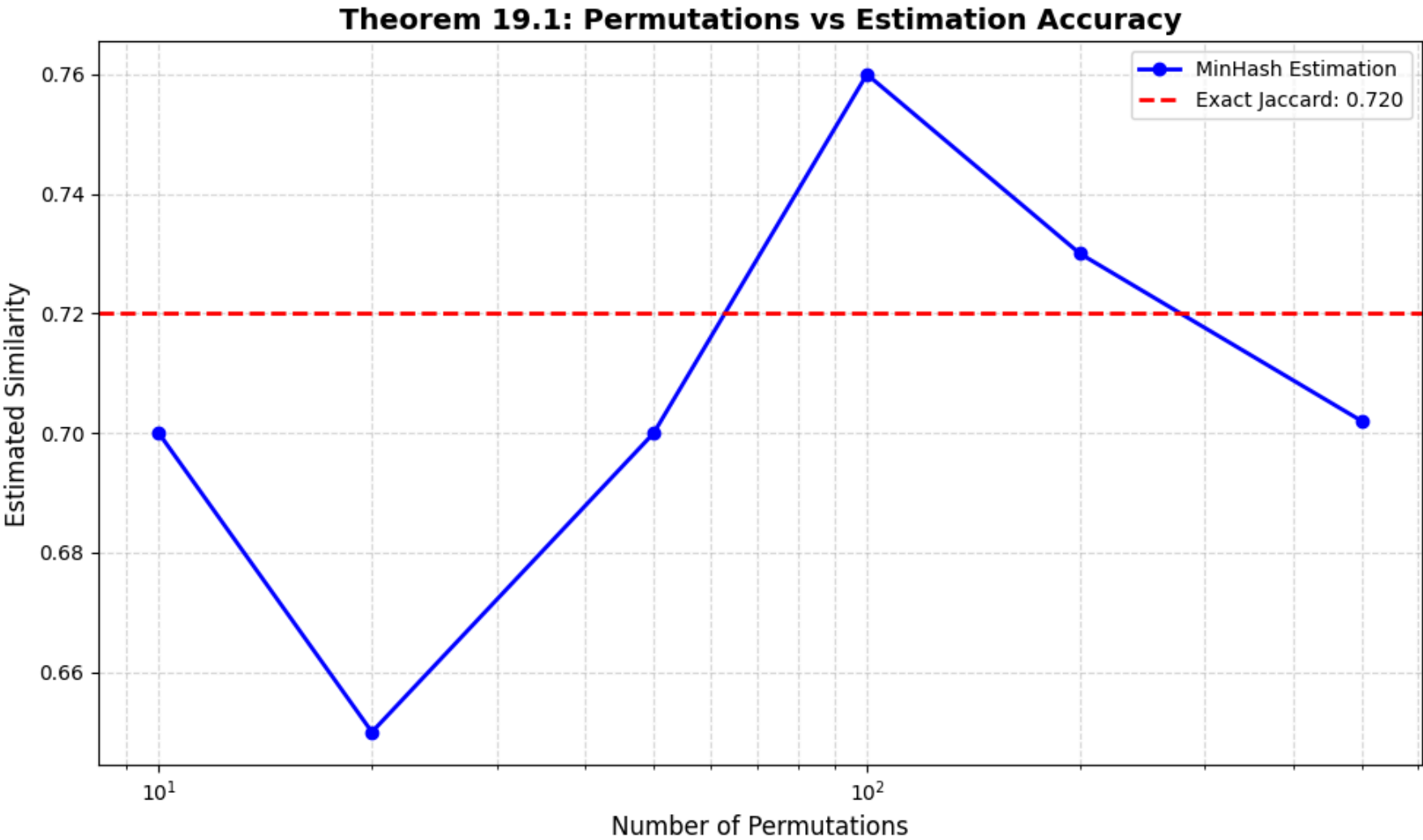
# اجرا
analyze_super_shingles(band_size=10)
```

```
# -----  
# نکات کلیدی آموزشی  
# -----  
  
print("\n" + "=" * 70)  
print("نکات کلیدی:")  
print("1. Shingling: تبدیل متن به مجموعه k-shingles (k-gramها)")  
print("2. = |AnB|/|AUB| ضریب جاکارد: معیار دقیق شباهت")  
print("3. MinHash: 200 permutation تخمین سریع جاکارد با (۱۹.۱ قضیه)")  
print("4. ها، خطا کاهش می‌یابدpermutation دقت: با افزایش")  
print("5. Super-Shingles: ۹۹٪ مقایسه‌ها را حذف می‌کند فیلتر اولیه،")  
print("6. کاربرد: گوگل و بینگ برای حذف ۴۰٪ صفحات تقریباً تکراری استفاده می‌کنند")  
print("7. Trade-off: و سرعت (band size) permutation (تعداد) بین دقت")
```

MinHash و Shingling تشخیص صفحات تقریباً تکراری با
=====

مقایسه خبر اصلی و خبر با تاریخ اضافی:
28 vs تعداد کلمات: 21
(|AnB|/|AUB|) جاکارد دقیق: 0.720
MinHash تخمین: 0.730 (200 permutation)
نتیجه: ✓ تقریباً تکراری

مقایسه خبر اصلی و خبر متفاوت:
جاکارد دقیق: 0.000
MinHash تخمین: 0.000
نتیجه: X متفاوت



ها بر دقت (قضیه ۱۹.۱) permutation تاثیر تعداد:
=====

```
10 permutations: estimate=0.700, error=2.78%  
20 permutations: estimate=0.650, error=9.72%  
50 permutations: estimate=0.700, error=2.78%  
100 permutations: estimate=0.760, error=5.56%  
200 permutations: estimate=0.730, error=1.39%  
500 permutations: estimate=0.702, error=2.50%
```

=====

Super-Shingles بهینه‌سازی با:
در هر متن: 20 Super-Shingles تعداد
مشترک (خبر ۱ و ۲): 1 Super-Shingles
مشترک (خبر ۱ و ۳): 0 Super-Shingles

Super-Shingles نتیجه فیلتر:
MinHash مشترک → بررسی دقیق با Super-Shingle خبر ۱ و ۲: دارای ✓
مشترک → بدون بررسی دقیق رد می‌شود Super-Shingle خبر ۱ و ۳: بدون X

💡 جفت‌های غیرمشابه را بدون محاسبه دقیق رد می‌کند ۹۹٪ Super-Shingles نکته

- =====
- نکات کلیدی:
1. Shingling: تبدیل متن به مجموعه k-shingles (k-gramها)
 2. = |AnB|/|AUB| ضریب جاکارد: معیار دقیق شباهت
 3. MinHash: 200 permutation تخمین سریع جاکارد با (۱۹.۱ قضیه)
 4. ها، خطا کاهش می‌یابدpermutation دقت: با افزایش
 5. Super-Shingles: ۹۹٪ مقایسه‌ها را حذف می‌کند فیلتر اولیه،
 6. کاربرد: گوگل و بینگ برای حذف ۴۰٪ صفحات تقریباً تکراری استفاده می‌کنند
 7. Trade-off: و سرعت (band size) permutation (تعداد) بین دقت

جمع‌بندی فصل ۱۹: مبانی جستجوی وب

چگونه می‌توان در یک محیط بی‌نهایت، پراکنده و شلوغ و در حال تغییر دائمی، اطلاعات مرتبط را پیدا کرد؟ فصل ۱۹ ما را به دنیای پیچیده و منحصر به فرد ****جستجوی وب**** می‌برد؛ جایی که در آن، نه تنها فناوری بازیابی اطلاعات، بلکه اقتصاد، روانشناسی کاربران و حتی نبردهای تجاری در کار است. این فصل نشان می‌دهد که چرا جستجوی وب چیزی فراتر از یک «کتابخانه دیجیتال» بزرگ است و چه چالش‌های منحصر به فردی آن را از بازیابی اطلاعات سنتی متمایز می‌کند.

- با ****پیشینه و تاریخچه وب**** آشنا شدیم. وب با سه ویژگی اصلی تعریف می‌شود: مقیاس بی‌سابقه، عدم وجود مرکزیت و تنوع فوق‌العاده در انگیزه‌های نویسندگان. این ویژگی‌ها وب را به یک «غرب وحشی» اطلاعات تبدیل کرد که مدیریت آن نیازمند رویکردهای کاملاً جدید بود.
- ساختار ****گراف وب (Web Graph)**** را به عنوان مدل بنیادی برای درک روابط بین صفحات بررسی کردیم. هر صفحه یک گره (گره) و هر پیوند یک یال (لبه) است. با استفاده از این مدل، مفاهیمی مانند ****توانگرایی (شاخص رتبه صفحه PageRank)**** و ساختار ****کمان و پاپیون (Bowtie)**** قابل توضیح هستند.
- با ****چالش هرزنامه‌نگاری (Spam)**** روبرو شدیم. انگیزه‌های تجاری باعث به وجود محتوای دستکاری شده برای فریب دادن به موتورهای جستجو شده است. تکنیک‌هایی مانند ****پنهان‌کاری (Cloaking)**** که در آن یک صفحه متفاوت به ربات جستجو و کاربر نمایش داده می‌شود، نمونه‌ای از این نبردهای هوشمندانه است.
- مدل ****اقتصادی و تجارت الکترونیک وب**** را بررسی کردیم. مدل‌های اولیه مبتنی بر نمایش آگهی (CPM) بودند، اما امروز مدل‌های مبتنی بر ****هزینه به ازای هر کلیک (CPC)**** و ****جستجوی حمایت‌شده (Sponsored Search)**** غالب هستند. این مدل‌ها به شرکت‌ها اجازه می‌دهند تا مستقیماً در مقابل کاربرانی که به دنبال محصولات یا خدماتشان هستند، ظاهر شوند.
- با ****تجربه کاربر در جستجوی وب**** آشنا شدیم. برخلاف کاربران حرفه‌ای پایگاه‌های اطلاعاتی، کاربران وب معمولاً جستارهای کوتاه و بدون عملگرهای پیچیده انجام می‌دهند. درک نیازهای آن‌ها به سه دسته اصلی تقسیم می‌شوند: ****اطلاعاتی**** (مانند جستجوی «لوکمی خون»)، ****ناوبری**** (مانند جستجوی نام یک وب‌سایت خاص)، و ****تراکنشی یا معاملتی**** (مانند جستجوی «خرید بلیط هوایی»).
- در نهایت، به چالش ****اندازه‌گیری نمایه (Index Size Estimation)**** و ****تشخیص نسخه‌های نزدیک به هم (Near-Duplicates)**** با استفاده از تکنیکی ****شینگلینگ (Shingling)**** پرداختیم این تکنیک‌ها برای مدیریت حجم عظیم وب و جلوگیری از ایندکس‌سازی محتوای تکراری ضروری هستند.

بیاید این مفاهیم را با چند مثال واقعی و کاربردی زنده کنیم:

ساختار گراف وب و توانگرایی: تصور کنید هر وب‌سایت یک شهر کوچک و هر پیوند یک جاده یک‌طرفه است. وب‌سایت‌های بزرگی مانند ویکی‌پدیا یا خبرگزاری‌های معتبر، «میادین‌های» بزرگی هستند که تعداد زیادی جاده به آن‌ها وارد می‌شود (in-degree بالا). موتورهای جستجو مانند گوگل از این مفهوم استفاده می‌کنند و به صفحاتی که از منابع معتبر زیادی لینک گرفته‌اند، امتیاز بالاتری می‌دهند. این همان منطق ****توانگرایی (شاخص رتبه صفحه PageRank)**** است: یک صفحه تا زمانی معتبر است که از صفحات معتبر دیگر به آن ارجاع داده شود.

💰 **هرزنامه‌نگاری و پنهان‌کاری:** یک آگهی املاک در «گلف کوچ مائویی» را در نظر بگیرید. یک هرزنامه‌نگار ممکن است وب‌سایتی بسازد که عبارت «گلف کوچ مائویی املاک» را صدها بار تکرار کند تا در نتایج جستجوی این عبارت رتبه بالایی بگیرد. وقتی کاربر روی این لینک کلیک می‌کند، به صفحه‌ای کاملاً متفاوت هدایت می‌شود که تبلیغات گران‌قیمت‌تری دارد. اما یک موتور جستجو ممکن است با استفاده از تکنیک ****پنهان‌کاری (Cloaking)**** متوجه این تقلب شود. در این تکنیک، سرور وب‌سایت به ربات جستجو یک صفحه با محتوای مرتبط با گلف (که برای رتبه بالا بهینه شده) نمایش می‌دهد، اما وقتی یک کاربر واقعی روی همان لینک کلیک می‌کند، او را به صفحه‌ای با تبلیغات و پیشنهادهای خرید هدایت می‌دهد. این مانند این است که یک مغازه‌دار دو چهره متفاوت به بازرس و مشتری نشان دهد!

👤 **مدل تجاری و جستجوی حمایت‌شده:** یک فروشگاه کوچک آنلاین که «کیف چرمی دست‌دوز» می‌فروشد، می‌خواهد وقتی کاربری عبارت «خرید کیف چرمی» را جستجو می‌کند، در بالاترین نتایج ظاهر شود. در مدل ****جستجوی حمایت‌شده****، این فروشگاه می‌تواند به موتور جستجو (مانند گوگل) مبلغی پرداخت کند تا وقتی کاربر این عبارت را جستجو می‌کند، آگهی فروشگاه او در بالای لیست (معمولاً با علامت «Ad») نمایش داده شود. اگر کاربر روی آگهی کلیک کند، فروشگاه هزینه‌ای اندک (معمولاً چند سنت) به موتور جستجو می‌پردازد. این مدل به کاربران اجازه می‌دهد تا خدمات مرتبط را پیدا کنند و به کسب‌وکارها اجازه می‌دهد تا دقیقاً در مقابل مشتریان بالقوه ظاهر شوند.

🔍 **اندازه‌گیری نمایه وب:** چگونه می‌توان تخمین زد که گوگل چند میلیارد صفحه را ایندکس کرده است؟ این کار مستقیم نیست، زیرا وب دائماً در حال تغییر است و صفحات پویا (Dynamic Pages) بی‌نهایت هستند. یک روش ساده، ****نمونه‌گیری مجدد (Capture-Recapture)****، این است: یک موتور جستجو ۱۰۰۰ صفحه تصادفی از وب برمی‌دارد و «علامت‌گذاری» می‌کند (مثلاً با افزودن یک رشته خاص به URL). سپس چند ماه بعد، دوباره ۱۰۰۰ صفحه تصادفی برمی‌دارد و بررسی می‌کند که چند مورد از صفحات علامت‌گذاری‌شده در نمونه جدید وجود دارند. اگر از ۱۰۰۰ صفحه اولیه، ۱۰ صفحه در نمونه دوم وجود داشته باشد، می‌توان تخمین زد که نمایه موتور جستجو حدوداً ۱۰ برابر کل وب است. این روش ساده، اما به ما درکی از مقیاس وب کمک می‌کند.

✂️ **تشخیص نسخه‌های نزدیک به هم با شینگلینگ:** دو خبرگزاری مختلف، یکی در تهران و دیگری در نیویورک، ممکن است یک گزارش کامل از یک بازی فوتبال را پوشش دهند. متن اصلی گزارش در هر دو سایت تقریباً یکسان است، اما نام خبرنگار، تاریخ و برخی جزئیات کوچک متفاوت است. الگوریتم ****شینگلینگ (Shingling)**** به جای مقایسه کل متن، فقط «اثر انگشتان» (شینگل‌ها) را مقایسه می‌کند. برای مثال، هر ۴ کلمه متوالی در متن یک شینگل تشکیل می‌شود. اگر دو مقاله تعداد زیادی شینگل مشترک داشته باشند، به احتمال زیاد نسخه‌ای از یکدیگر هستند. این روش به موتورهای جستجو اجازه می‌دهد تا با ذخیره کردن فقط یکی از نسخه‌ها در نمایه خود، از اتلاف حافظه و پردازش بیهوده جلوگیری کنند.

این فصل نشان داد که جستجوی وب یک حوزه چندرشته‌ای است که در آن، فناوری اطلاعات با اقتصاد دیجیتال و رفتار کاربران در هم می‌آمیزد. درک موفقیت یک موتور جستجوی مدرن نه تنها در بهینه‌سازی الگوریتم‌های بازیابی، بلکه در درک این اکوسیستم پیچیده و طراحی سیستم‌هایی است که بتوانند با هرزنامه‌نگاری هوشمندانه، مدل‌های تجاری منصفانه و تجربه کاربری بهینه مقابله کنند.